

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



ĐỒ ÁN MÔN HỌC KỸ THUẬT MÁY TÍNH

**ỨNG DỤNG IOT TRONG GIÁM SÁT VÀ ĐIỀU
PHỐI VẬN CHUYỂN ĐẠN DƯỢC**

**CHUYÊN NGÀNH: KỸ THUẬT MÁY TÍNH
HỘI ĐỒNG: XX KỸ THUẬT MÁY TÍNH**

Giảng viên hướng dẫn:

TS. Võ Tuấn Bình - HCMUT

Sinh viên thực hiện:

Trương Nguyễn Hoàng Anh - 2210147

Hoàng Sỹ Xuân Sơn - 2212937

Lâm Hoàng Tân - 2213054

Thành phố Hồ Chí Minh, tháng 9 năm 2025

Mục lục

Danh sách hình vẽ	viii
Danh sách bảng	x
Lời cam kết	xiii
Lời cảm ơn	xv
Tóm tắt	xvii
1 Giới thiệu	1
1.1 Bối cảnh của đề tài	1
1.2 Mục tiêu nghiên cứu	2
1.2.1 Mục tiêu tổng quát	2
1.2.2 Mục tiêu cụ thể	2
1.3 Đối tượng và phạm vi nghiên cứu	2
1.3.1 Đối tượng nghiên cứu	2
1.3.2 Phạm vi triển khai của đề tài	2
1.4 Cấu trúc của đề tài	3
2 Cơ sở lý thuyết và các công nghệ liên quan	5
2.1 Tổng quan về Internet of Things	5
2.2 Các mô hình kiến trúc hệ thống IoT	6
2.2.1 Điện toán đám mây (Cloud Computing)	6
2.2.2 Điện toán biên (Edge Computing)	6
2.2.3 Kiến trúc lai (Hybrid Cloud-Edge)	6
2.3 Các công nghệ truyền thông không dây	7
2.3.1 LoRa	7
2.3.1.1 Giới thiệu	7
2.3.1.2 Nguyên lý hoạt động	8
2.3.1.3 Đặc điểm nổi bật	8

2.3.1.4	Ưu điểm và nhược điểm	9
2.3.2	4G/5G	9
2.3.2.1	Giới thiệu	9
2.3.2.2	Nguyên lý hoạt động	10
2.3.2.3	Đặc điểm nổi bật	10
2.3.2.4	Ưu điểm và nhược điểm	11
2.3.3	Mạng vệ tinh	11
2.3.3.1	Giới thiệu	11
2.3.3.2	Nguyên lý hoạt động	12
2.3.3.3	Đặc điểm nổi bật	12
2.3.3.4	Ưu điểm và nhược điểm	12
2.4	Các giao thức tầng ứng dụng	13
2.4.1	MQTT	13
2.4.1.1	Giới thiệu	13
2.4.1.2	Nguyên lý hoạt động	14
2.4.1.3	Đặc điểm nổi bật	14
2.4.1.4	Ưu điểm và nhược điểm	15
2.4.2	REST (HTTP)	16
2.4.2.1	Giới thiệu	16
2.4.2.2	Nguyên lý hoạt động	16
2.4.2.3	Đặc điểm nổi bật	17
2.4.2.4	Ưu điểm và nhược điểm	17
2.4.3	GraphQL	18
2.4.3.1	Giới thiệu	18
2.4.3.2	Nguyên lý hoạt động	18
2.4.3.3	Đặc điểm nổi bật	19
2.4.3.4	Ưu điểm và nhược điểm	19
2.4.4	So sánh và lựa chọn giao thức	20
2.4.4.1	So sánh các giao thức tầng ứng dụng	20
2.4.4.2	So sánh các công nghệ truyền thông không dây	20
2.4.4.3	So sánh các kiến trúc	21
2.4.5	Lựa chọn kiến trúc, giao thức và giải pháp tổng thể	23
2.5	Các loại cảm biến, thiết bị sử dụng	23
2.5.1	ESP32 NodeMCU-32S CH340 Ai-Thinker	23
2.5.2	Cảm biến nhiệt độ, độ ẩm DHT11	24
2.5.3	Cảm biến gia tốc 3 trục ADXL345	25
2.5.4	Module GPS NEO-6MV2	26

2.5.5	Module truyền thông LoRa Ra-08H Kit	27
3	Phân tích yêu cầu hệ thống	29
3.1	Yêu cầu chức năng	29
3.1.1	Người dùng	29
3.1.2	Hệ thống	30
3.2	Yêu cầu phi chức năng	30
3.2.1	Hiệu năng	30
3.2.2	Tính sẵn sàng	30
3.2.3	Khả năng mở rộng	31
3.2.4	Tính dễ sử dụng	31
3.2.5	Tính tương thích	31
3.2.6	Khả năng bảo trì	31
3.3	Quy trình vận chuyển đạn dược	31
3.4	Các rủi ro cần giám sát	32
3.5	Use-case	33
3.5.1	Use-case diagram	33
3.5.2	Đặc tả use-case	34
3.5.2.1	Use-case 1: Điều phối lộ trình	34
3.5.2.2	Use-case 2: Nhận lệnh vận chuyển	35
3.5.2.3	Use-case 3: Cập nhật trạng thái vận chuyển	35
3.5.2.4	Use-case 4: Giám sát vận chuyển	36
3.5.2.5	Use-case 5: Phát hiện bất thường và gửi cảnh báo	36
3.6	Activity diagram	37
3.6.1	Điều phối lộ trình	37
3.6.2	Giám sát & cập nhật trạng thái vận chuyển	38
3.7	Sequence diagram	40
3.7.1	Điều phối lộ trình	41
3.7.2	Giám sát & cập nhật trạng thái vận chuyển	43
4	Thiết kế hệ thống	45
4.1	Sơ đồ kiến trúc tổng thể	45
4.2	Sơ đồ kết nối phần cứng	47
4.3	Thiết kế truyền thông dữ liệu	48
4.4	Thiết kế ứng dụng giám sát	49
4.4.1	Backend	50
4.4.2	Frontend	50
4.5	Thiết kế giao diện người dùng	51

5	Cài đặt và triển khai hệ thống	55
5.1	Cài đặt môi trường phát triển	55
5.1.1	Môi trường phát triển phần cứng	55
5.1.1.1	Môi trường phát triển firmware LoRa P2P	55
5.1.2	Môi trường phát triển phần mềm	56
5.2	Cài đặt firmware cho Node Lora P2P	56
5.2.1	Các bước build firmware LoRa P2P	56
5.2.2	Quá trình phát triển firmware LoRa P2P	57
5.2.2.1	Node TX (Truyền tin)	57
5.2.2.2	Node RX (Gateway nhận tin)	58
5.3	Cài đặt backend và xử lý dữ liệu	59
5.3.1	Thiết lập hạ tầng MQTT Broker	59
5.3.2	Triển khai dịch vụ cầu nối (Bridge Service)	60
5.4	Cài đặt và phát triển ứng dụng di động	60
5.4.1	Kiến trúc ứng dụng Flutter	60
5.4.2	Cơ chế đồng bộ dữ liệu thời gian thực	61
5.5	Kết nối và tích hợp thiết bị	61
5.5.1	Sơ đồ kết nối Node TX (Sensor Node)	61
5.5.2	Sơ đồ kết nối Node RX (Gateway Node)	62
5.5.3	Tích hợp dữ liệu (Data Integration)	62
5.6	Quy trình vận hành và kiểm thử hệ thống	63
6	Kết quả và đánh giá	65
6.1	Các kịch bản thử nghiệm	65
6.1.1	Kịch bản kiểm thử khoảng cách truyền thông (Range Test)	65
6.1.1.1	Kết quả thử nghiệm	65
6.1.1.2	Phân tích và Đánh giá	66
6.1.2	Kịch bản kiểm thử chức năng cảnh báo	67
6.1.2.1	Kết quả thử nghiệm	68
6.1.2.2	Phân tích và đánh giá	68
6.2	Kiểm thử tích hợp và Phân tích độ trễ	68
6.2.1	Phân rã độ trễ toàn trình (Latency Breakdown)	68
6.2.2	Thử nghiệm mô phỏng vận chuyển thực tế	68
6.3	Đánh giá kết quả đạt được	69
7	Kết luận và hướng phát triển	71
7.1	Đánh giá chung	71
7.2	Hạn chế	72

7.3 Hướng phát triển	72
Tài liệu tham khảo	74

Danh sách hình vẽ

2.3.1 Minh hoạ về công nghệ LoRa	8
2.4.1 Minh hoạ về nguyên lý hoạt động của MQTT	14
2.5.1 ESP32 NodeMCU-32S CH340 Ai-Thinker	24
2.5.2 Cảm biến nhiệt độ, độ ẩm DHT11	25
2.5.3 Cảm biến gia tốc 3 trục ADXL345	26
2.5.4 Module GPS NEO-6MV2	27
2.5.5 Module LoRa Ra-08H Development Board Ai-Thinker	28
3.5.1 Usecase diagram	33
3.6.1 Activity diagram: Điều phối lộ trình	38
3.6.2 Activity diagram: Giám sát & cập nhật trạng thái vận chuyển	40
3.7.1 Sequence diagram: Điều phối lộ trình	42
3.7.2 Sequence diagram: Giám sát & cập nhật trạng thái vận chuyển	44
4.1.1 Sơ đồ khối của toàn bộ hệ thống	46
4.2.1 Sơ đồ kết nối phần cứng của hệ thống	47
4.3.1 Mô phỏng luồng dữ liệu tổng quan của hệ thống	48
4.5.1 Các giao diện liên quan đến phần đăng nhập, đăng ký	51
4.5.2 Các giao diện liên quan đến trang Home	52
4.5.3 Các giao diện liên quan đến trang Maps	52
4.5.4 Các giao diện liên quan đến trang Dashboard	53
4.5.5 Các giao diện liên quan đến trang Profile	53
6.2.1 Giao diện ứng dụng di động trong quá trình thử nghiệm thực tế	69

Danh sách bảng

2.4.1 So sánh các giao thức tầng ứng dụng	20
2.4.2 So sánh các công nghệ truyền thông không dây	21
2.4.3 So sánh các kiến trúc hệ thống IoT	22
2.5.1 Thông số kỹ thuật của ESP32 NodeMCU-32S CH340 Ai-Thinker	24
2.5.2 Thông số kỹ thuật của cảm biến nhiệt độ, độ ẩm DHT11 . . .	25
2.5.3 Thông số kỹ thuật của cảm biến gia tốc 3 trục ADXL345 . . .	25
2.5.4 Thông số kỹ thuật của module GPS NEO-6MV2	26
2.5.5 Thông số kỹ thuật của Kit LoRa Ra-08H Ai-Thinker	27
3.5.1 Đặc tả Use-case UC02: Điều phối lộ trình	34
3.5.2 Đặc tả Use-case UC03: Nhận lệnh vận chuyển	35
3.5.3 Đặc tả Use-case UC04: Cập nhật trạng thái vận chuyển	35
3.5.4 Đặc tả Use-case UC05: Giám sát vận chuyển	36
3.5.5 Đặc tả Use-case UC06: Phát hiện bất thường và gửi cảnh báo .	36
5.5.1 Sơ đồ chân kết nối phần cứng Node TX	61
5.5.2 Sơ đồ chân kết nối phần cứng Node RX	62
5.6.1 Tổng hợp chức năng và vai trò của các thành phần trong hệ thống	64
6.1.1 Kết quả đo đạc Range Test tại KTX Khu A	66
6.2.1 Phân rã độ trễ xử lý dữ liệu của hệ thống	68

LỜI CAM KẾT

LỜI CẢM ƠN

TÓM TẮT

1

GIỚI THIỆU

Chương này trình bày tổng quan về đề tài, bao gồm bối cảnh, các vấn đề thực tiễn đặt ra và lý do cần thiết phải xây dựng hệ thống giám sát vận chuyển đạn được ứng dụng IoT. Bên cạnh đó, chương cũng nêu rõ mục tiêu nghiên cứu, phạm vi triển khai và cấu trúc tổng thể của báo cáo. Những nội dung này đóng vai trò định hướng cho toàn bộ đề tài, giúp người đọc hiểu được mục đích, giới hạn và cách thức tổ chức của các phần tiếp theo.

1.1 Bối cảnh của đề tài

Trong bối cảnh hiện nay, nhu cầu hiện đại hóa các hoạt động quản lý và đảm bảo an toàn trong lĩnh vực quân sự ngày càng trở nên cấp thiết. Trong đó, công tác vận chuyển đạn được là một quy trình có độ rủi ro cao, đòi hỏi sự giám sát chặt chẽ về vị trí, tình trạng môi trường và an toàn kỹ thuật trong suốt quá trình di chuyển. Tuy nhiên, phần lớn hoạt động theo dõi hiện nay vẫn phụ thuộc vào ghi chép thủ công hoặc các phương pháp quản lý truyền thống, dẫn đến hạn chế trong khả năng giám sát theo thời gian thực, chậm trễ trong xử lý sự cố và thiếu tính minh bạch trong quản lý hành trình.

Sự phát triển của IoT đã mở ra khả năng tự động hóa giám sát, thu thập dữ liệu liên tục và truyền tải thông tin theo thời gian thực. IoT cho phép tích hợp các cảm biến đo rung, nhiệt độ, vị trí và các tín hiệu khác vào một hệ thống tập trung, hỗ trợ đơn vị quản lý phát hiện bất thường và đưa ra cảnh báo kịp thời. Điều này góp phần nâng cao tính an toàn, độ tin cậy và hiệu quả trong quy trình vận chuyển đạn được, vốn là một hoạt động đặc biệt nhạy cảm và quan trọng.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Nghiên cứu và xây dựng hệ thống giám sát vận tải quân sự dựa trên công nghệ LoRa P2P (Point-to-Point) và vi điều khiển ESP32, đảm bảo khả năng truyền tin tin cậy trong điều kiện thiếu hạ tầng mạng.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát nêu trên, đề tài tập trung giải quyết ba nhiệm vụ cốt lõi sau:

Thứ nhất, **nghiên cứu và tối ưu hóa kiến trúc phần mềm nhúng** cho giao thức truyền tin LoRa. Trọng tâm là đảm bảo tính thời gian thực (real-time) và tối ưu hóa năng lượng tiêu thụ cho thiết bị đầu cuối hoạt động bằng pin.

Thứ hai, **thiết kế khối thu thập dữ liệu** tích hợp đa cảm biến. Hệ thống sẽ sử dụng cảm biến DHT11 để giám sát nhiệt độ, độ ẩm; module Neo-6M cho định vị toạ độ GPS; và gia tốc kế ADXL345 nhằm phát hiện các va đập bất thường có thể ảnh hưởng đến an toàn của khí tài.

Thứ ba, **xây dựng cơ sở hạ tầng Back-end và giao diện Frontend**. Hệ thống cần có khả năng thu thập dữ liệu tập trung thông qua giao thức MQTT và hiển thị trực quan các thông số giám sát trên Dashboard (sử dụng nền tảng Firebase và Web App) để phục vụ công tác chỉ huy, điều hành.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu sâu vào hai nhóm đối tượng chính:

- **Về phần cứng:** Vi điều khiển ESP32 đóng vai trò bộ xử lý trung tâm, Module LoRa RA-08H cho truyền thông tầm xa, và các module cảm biến ngoại vi (GPS, DHT11, ADXL345).
- **Về giải pháp phần mềm:** Các giao thức truyền thông điệp tin cậy (LoRa P2P, MQTT), kỹ thuật lập trình nhúng trên Arduino IDE, ngôn ngữ Python cho các tác vụ cầu nối (bridge), và cơ sở dữ liệu thời gian thực Firebase.

1.3.2 Phạm vi triển khai của đề tài

Trong khuôn khổ của đồ án tốt nghiệp, phạm vi nghiên cứu được giới hạn ở các nội dung sau:

1. **Môi trường phát triển:** Tập trung phát triển và kiểm thử trên bộ Kit phần

cứng mô phỏng, chưa triển khai trên phương tiện vận tải thực tế quy mô lớn.

1

2. **Kịch bản thử nghiệm:** Thực hiện đo đặc khả năng truyền tin trong phạm vi hẹp (phòng thí nghiệm) và mô phỏng các kịch bản mất kết nối để đánh giá tính năng lưu trữ cục bộ (Local Logging) trên thẻ nhớ SD.
3. **Giới hạn nghiên cứu:** Đề tài tập trung giải quyết bài toán ở tầng ứng dụng (Application Layer) và tầng truyền thông (Network Layer), chưa đi sâu vào các giải pháp bảo mật vật lý chuyên dụng hay thiết kế cơ khí chịu lực cho vỏ hộp thiết bị.

1.4 Cấu trúc của đề tài

Đề tài được tổ chức thành bảy chương theo trình tự logic giúp người đọc dễ dàng theo dõi quá trình nghiên cứu và triển khai hệ thống. Mỗi chương đảm nhận một vai trò riêng, từ việc giới thiệu bối cảnh cho đến trình bày lý thuyết nền tảng, phân tích yêu cầu, thiết kế, cài đặt, đánh giá và tổng kết. Cấu trúc này đảm bảo tính mạch lạc và phản ánh đầy đủ các bước thực hiện của đồ án.

Chương 1 - Giới thiệu: Tổng quan về đề tài, lý do chọn đề tài và mục tiêu nghiên cứu.

Chương 2 - Cơ sở lý thuyết: Trình bày về công nghệ LoRa, vi điều khiển ESP32 và các giao thức liên quan.

Chương 3 - Phân tích yêu cầu: Xác định các yêu cầu phi chức năng và chức năng của hệ thống.

Chương 4 - Thiết kế hệ thống: Thiết kế kiến trúc phần cứng và phần mềm, sơ đồ khối và luồng dữ liệu.

Chương 5 - Cài đặt và Triển khai: Mô tả quá trình lập trình, lắp ráp mạch và cấu hình hệ thống.

Chương 6 - Kết quả và Đánh giá: Trình bày kết quả thử nghiệm thực tế và đánh giá hiệu năng.

Chương 7 - Kết luận và Hướng phát triển: Tổng kết các kết quả đạt được và đề xuất hướng nghiên cứu tiếp theo.

CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ LIÊN QUAN

Chương này trình bày cơ sở lý thuyết và các công nghệ liên quan phục vụ việc xây dựng hệ thống IoT. Nội dung bao gồm tổng quan về IoT, các giao thức truyền thông phổ biến, các loại cảm biến và phần cứng sử dụng, cùng với các kỹ thuật bảo mật cần thiết trong quá trình truyền tải và xử lý dữ liệu.

2.1 Tổng quan về Internet of Things

Internet of Things (IoT) là thuật ngữ dùng để chỉ các đối tượng có thể được nhận biết cũng như sự tồn tại của chúng trong một kiến trúc mạng tính kết nối. Đây là một viễn cảnh trong đó mọi vật, mọi con vật hoặc con người được cung cấp các định danh và khả năng tự động truyền tải dữ liệu qua một mạng lưới mà không cần sự tương tác giữa con người với con người hoặc con người với máy tính. IoT tiến hoá từ sự hội tụ của các công nghệ không dây, hệ thống vi cơ điện tử và Internet.

"Thing" trong Internet of Things, có thể là một trang trại động vật với bộ tiếp sóng chip sinh học, một chiếc xe ô tô tích hợp các cảm biến để cảnh báo lái xe khi lốp quá non, hoặc bất kỳ đồ vật nào do tự nhiên sinh ra hoặc do con người sản xuất ra mà có thể được gán với một địa chỉ IP và được cung cấp khả năng truyền tải dữ liệu qua mạng lưới.

2.2 Các mô hình kiến trúc hệ thống IoT

Việc lựa chọn kiến trúc hệ thống đóng vai trò then chốt trong việc định hình hiệu năng, độ trễ và khả năng mở rộng của một giải pháp IoT. Hiện nay, có ba mô hình xử lý dữ liệu chính đang được áp dụng rộng rãi: Điện toán đám mây (Cloud Computing), Điện toán biên (Edge Computing) và Kiến trúc lai (Hybrid Cloud-Edge).

2

2.2.1 Điện toán đám mây (Cloud Computing)

Điện toán đám mây là mô hình tập trung hóa, trong đó toàn bộ dữ liệu từ các thiết bị cảm biến (Device Layer) được truyền trực tiếp về các máy chủ trung tâm (Cloud Server) để lưu trữ và xử lý. Ưu điểm lớn nhất của mô hình này là khả năng lưu trữ không giới hạn và sức mạnh tính toán khổng lồ, cho phép thực hiện các thuật toán phức tạp như Big Data hay Machine Learning trên tập dữ liệu lịch sử.

Tuy nhiên, đối với các ứng dụng yêu cầu phản hồi thời gian thực như giám sát vận chuyển hàng hóa nguy hiểm, mô hình Cloud thuần túy bộc lộ nhiều hạn chế. Độ trễ mạng (Latency) cao do khoảng cách truyền tải xa và sự phụ thuộc hoàn toàn vào kết nối Internet là những rào cản lớn. Trong trường hợp phương tiện di chuyển vào vùng mất sóng, dữ liệu sẽ bị gián đoạn, dẫn đến việc mất khả năng giám sát và cảnh báo tức thời.

2.2.2 Điện toán biên (Edge Computing)

Trái ngược với Cloud, điện toán biên di dời khả năng xử lý tính toán từ trung tâm mạng xuống các thiết bị ở rìa mạng (Edge Devices), gần với nguồn sinh dữ liệu. Trong mô hình này, các Gateway hoặc các nút cảm biến thông minh sẽ trực tiếp thu thập, lọc nhiễu và xử lý dữ liệu sơ bộ trước khi quyết định có gửi đi hay không.

Điện toán biên giải quyết triệt để bài toán về độ trễ và băng thông. Các cảnh báo nguy cấp (như nhiệt độ vượt ngưỡng, va đập mạnh) được xử lý và phát ra ngay tại chỗ (Local Alert) mà không cần chờ phản hồi từ máy chủ. Điều này đặc biệt quan trọng trong vận chuyển đạn dược, nơi mili-giây phản hồi có thể quyết định sự an toàn. Mặc dù vậy, thiết bị biên thường bị giới hạn về năng lượng và khả năng lưu trữ, không phù hợp để quản lý dữ liệu dài hạn.

2.2.3 Kiến trúc lai (Hybrid Cloud-Edge)

Nhận thấy ưu và nhược điểm riêng biệt của hai mô hình trên, kiến trúc lai Hybrid Cloud-Edge ra đời như một giải pháp dung hòa tối ưu. Trong kiến trúc

này, quy trình xử lý dữ liệu được phân chia thành hai tầng rõ rệt:

- **Tầng biên (Edge Layer):** Đảm nhận các tác vụ yêu cầu thời gian thực (Real-time processing). Thiết bị tại biên (ESP32 Gateway) sẽ thu thập dữ liệu cảm biến, thực hiện lọc nhiễu (ví dụ: thuật toán Kalman hoặc trung bình trượt) và đưa ra các quyết định cảnh báo tức thời nếu phát hiện bất thường. Đồng thời, nó đóng gói và nén dữ liệu để tối ưu hóa băng thông truyền tải.
- **Tầng đám mây (Cloud Layer):** Đóng vai trò là kho lưu trữ trung tâm (Centralized Storage) và trung tâm điều phối. Dữ liệu đã qua xử lý từ tầng biên được đồng bộ lên Cloud để lưu trữ dài hạn, phục vụ cho việc hiển thị Dashboard, phân tích xu hướng và trích xuất báo cáo hậu cần.

Dựa trên yêu cầu đặc thù của đề tài về việc giám sát vận chuyển đạn dược – một bài toán đòi hỏi cả sự phản ứng tức thời tại hiện trường (Edge) và khả năng quản lý tập trung từ xa (Cloud), nhóm nghiên cứu quyết định lựa chọn Kiến trúc Hybrid Cloud-Edge làm nền tảng thiết kế hệ thống. Sự kết hợp này đảm bảo hệ thống vẫn có thể hoạt động cảnh báo cục bộ ngay cả khi mất kết nối mạng (Offline Mode), đồng thời vẫn duy trì được cái nhìn toàn cảnh cho người điều phối khi có kết nối trở lại.

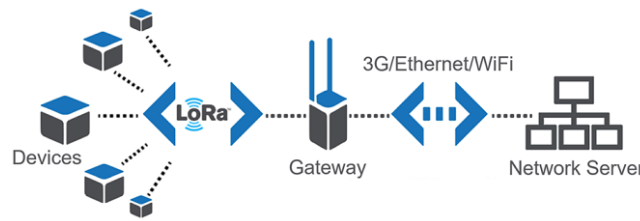
2.3 Các công nghệ truyền thông không dây

2.3.1 LoRa

2.3.1.1 Giới thiệu

LoRa, viết tắt của Long Range Radio, là một loại công nghệ hỗ trợ truyền dữ liệu trong những khoảng cách lên đến hàng chục km mà không cần thêm bất kỳ các mạch khuếch đại công suất nào. LoRa giúp việc truyền và nhận dữ liệu trở nên đơn giản hơn, tiết kiệm năng lượng tiêu thụ hiệu quả.

Một mạng LoRa có thể cung cấp vùng phủ sóng tương tự như của một mạng di động. Trong một số trường hợp, các antenna Lora có thể kết hợp với antenna di động khi các tần số là gần nhau, từ đó giúp tiết kiệm đáng kể chi phí.



Hình 2.3.1: Minh họa về công nghệ LoRa

2.3.1.2 Nguyên lý hoạt động

Nền tảng phát triển công nghệ LoRa dựa trên kỹ thuật điều chế Chirp Spread Spectrum. Khi các dữ liệu được tạo xung với tần số cao để tạo ra những tín hiệu có dải tần cao hơn. Các tín hiệu này sẽ được mã hóa theo các chuỗi chirp signal (tín hiệu hình sin thay đổi theo thời gian) trước khi được gửi đi từ antenna. Có hai loại chirp signal, bao gồm tần số up-chirp theo thời gian và tần số của down-chirp giảm dần theo thời gian.

Nguyên tắc hoạt động này hỗ trợ thiết bị giảm độ phức tạp và tăng độ chính xác cần thiết cho mạch nhận để có thể giải mã và điều chỉnh lại dữ liệu. LoRa không yêu cầu nhiều công suất phát mà vẫn có thể truyền đi xa, vì tín hiệu LoRa có thể nhận được ở khoảng cách xa ngay cả khi cường độ tín hiệu thấp hơn nhiều xung quanh.

Băng tần hoạt động của LoRa nằm trong khoảng từ 430MHz đến 915MHz, áp dụng cho các khu vực khác nhau trên thế giới. Tín hiệu chirp sẽ cho phép các tín hiệu LoRa hoạt động trong cùng một khu vực mà không gây nhiễu lẫn nhau, cho phép nhiều thiết bị trao đổi dữ liệu trên nhiều kênh đồng thời.

2.3.1.3 Đặc điểm nổi bật

Là một công nghệ hiện đại được sử dụng phổ biến hiện nay, LoRa có khả năng truyền dữ liệu ở khoảng cách cực xa và có thể đạt khoảng cách truyền hơn 15km trong môi trường mở hoặc rộng hơn nữa. Nó còn có thể chạy với mức tiêu thụ điện năng thấp, điều này có thể kéo dài tuổi thọ pin và giảm chi phí sử dụng khi không cần thay quá nhiều lần.

Với kỹ thuật truyền của công nghệ LoRa, tốc độ truyền tuy thấp nhưng vẫn cung cấp đủ băng thông cho một số ứng dụng IoT nhất định, chẳng hạn như định vị, theo dõi tài nguyên và gửi thông tin trạng thái. Công nghệ này có khả năng chống nhiễu tốt và khả năng tự động tìm kiếm kênh truyền tốt nhất, giúp đảm bảo tính toàn vẹn của dữ liệu truyền đi.

2.3.1.4 Ưu điểm và nhược điểm

Nổi bật nhất, LoRa có mức tiêu thụ điện năng thấp, đây là ưu điểm lớn nhất của công nghệ LoRa. Bởi mức tiêu thụ điện năng của công nghệ này thấp. Tương ứng, tuổi thọ của ắc quy có thể lên đến 10 năm, hỗ trợ các nhà máy, doanh nghiệp giảm chi phí thay thế ắc quy.

LoRa còn có thể hỗ trợ máy tính truyền dữ liệu vài km mà không cần bộ khuếch đại công suất. Do LoRa sử dụng ít nhiễu điện từ hơn nên tín hiệu có thể duy trì khoảng cách xa hoặc khả năng làm việc mạnh mẽ ngay cả trong môi trường đô thị với những ngôi nhà dày đặc.

LoRa là một giao thức mạng mở, có khả năng cung cấp các kết nối nút cuối được tiêu chuẩn hóa giữa những máy tính và thiết bị IoT. Điều này cho phép mỗi nhà máy nhanh chóng triển khai các ứng dụng IoT ở mọi nơi.

Công nghệ còn sở hữu mã hóa AES128, cho phép xác thực lẫn nhau, đảm bảo tính toàn vẹn và tăng tính bảo mật.

Tuy LoRa là một công nghệ được ưa chuộng sử dụng nhưng nó không phải một công nghệ có tính hoàn hảo về mọi mặt. Công nghệ này không phù hợp với những công việc cần tải dữ liệu lớn, đây cũng là nhược điểm lớn nhất đối với công nghệ LoRa. Do các sóng truyền ở tần số này làm chậm tốc độ truyền và tải trọng của công nghệ bị giới hạn ở 100 byte. Do đó, độ trễ của công nghệ LoRa sẽ cao hơn các phương pháp khác.

Khi sử dụng công nghệ LoRa, người dùng sẽ gặp khó khăn trong việc lắp đặt các gateway trong khu vực nội thành cũng là một trở ngại cho việc phổ cập công nghệ LoRa tại các khu vực đông dân cư.

LoRa có khả năng truyền dữ liệu hạn chế và không phù hợp với các ứng dụng yêu cầu truyền dữ liệu lớn. Ngoài ra, để có thể triển khai một hệ thống LoRa hoàn chỉnh, cần có nhiều cổng và thiết bị kết nối, điều này làm tăng chi phí triển khai.

2.3.2 4G/5G

2.3.2.1 Giới thiệu

Mạng 4G và 5G là các thế hệ mạng di động băng thông rộng được sử dụng rộng rãi trong các hệ thống IoT yêu cầu tốc độ truyền dữ liệu cao, độ trễ thấp và phạm vi phủ sóng lớn.

Mạng 4G cung cấp tốc độ truyền tải cao hơn nhiều so với 3G, hỗ trợ tốt các ứng dụng IoT như camera giám sát, theo dõi phương tiện hoặc truyền dữ liệu thời gian thực. Mạng 5G là thế hệ mạng mới nhất với tốc độ vượt trội, độ trễ

cực thấp, khả năng kết nối số lượng lớn thiết bị, đặc biệt phù hợp với các ứng dụng IoT quy mô lớn như xe tự hành, thành phố thông minh hoặc tự động hoá công nghiệp.

2.3.2.2 Nguyên lý hoạt động

Mạng 4G và 5G đều hoạt động dựa trên kiến trúc mạng di động tổ ong, trong đó mỗi khu vực được bao phủ bởi các trạm gốc truyền nhận tín hiệu vô tuyến. Khi một thiết bị IoT hoặc thiết bị di động khởi tạo kết nối, tín hiệu sẽ được truyền đến trạm gốc gần nhất để thực hiện quá trình xác thực và thiết lập phiên làm việc với mạng lõi. Đối với 4G, mạng lõi EPC chịu trách nhiệm định tuyến gói dữ liệu và đảm bảo chất lượng truyền dẫn. Trong khi đó, 5G sử dụng kiến trúc hiện đại hơn, cho phép xử lý dữ liệu linh hoạt và hiệu quả thông qua các chức năng ảo hóa và tách rời.

Sau khi kết nối được thiết lập, dữ liệu IoT được truyền dưới dạng gói IP qua trạm gốc, đi qua mạng lõi và đến server hoặc nền tảng cloud. Công nghệ 5G còn sử dụng các kỹ thuật tiên tiến như beamforming và Massive MIMO để tập trung tín hiệu theo hướng tối ưu, tăng tốc độ và giảm nhiễu. Ngoài ra, 5G hỗ trợ phân đoạn mạng, cho phép tạo ra nhiều “lát cắt” mạng độc lập nhằm phục vụ các nhóm ứng dụng có yêu cầu khác nhau, từ băng thông cao, độ trễ thấp cho đến số lượng kết nối lớn. Nhờ những cơ chế hoạt động này, 4G và đặc biệt là 5G có khả năng cung cấp tốc độ truyền tải nhanh, độ trễ thấp và hiệu suất kết nối ổn định cho các hệ thống IoT hiện đại.

2.3.2.3 Đặc điểm nổi bật

Công nghệ 4G và 5G đều mang lại những bước tiến quan trọng trong lĩnh vực truyền thông không dây, đặc biệt đối với các hệ thống IoT hiện đại. Ở thế hệ 4G, tốc độ truyền tải dữ liệu được nâng lên đáng kể, đạt đến 100 Mbps đối với thiết bị di động và có thể lên tới 1 Gbps trong các kết nối cố định. Độ trễ cũng được cải thiện, giảm xuống chỉ còn 5–20 ms, giúp phản hồi nhanh hơn cho các ứng dụng thời gian thực. 4G sử dụng các công nghệ như OFDMA và MIMO để tăng băng thông, hỗ trợ nhiều thiết bị kết nối đồng thời, đảm bảo hiệu suất ổn định trong môi trường đông người dùng.

Bước sang 5G, công nghệ này mang đến những đột phá vượt trội về tốc độ, độ trễ và khả năng kết nối. Tốc độ tải xuống lý thuyết có thể đạt tới 10 Gbps, nhanh hơn 4G từ 10 đến 100 lần, đồng thời độ trễ giảm xuống mức cực thấp chỉ khoảng 1 ms. Điều này cho phép 5G hỗ trợ tốt các ứng dụng yêu cầu phản hồi

tức thời như xe tự hành, điều khiển robot từ xa, thực tế ảo. 5G cũng cho phép kết nối hàng triệu thiết bị trên mỗi km^2 , đáp ứng nhu cầu của đô thị thông minh và các hệ thống IoT mật độ cao. Ngoài ra, nhờ ứng dụng sóng milimet, Massive MIMO và công nghệ phân mảnh mạng, 5G có khả năng tối ưu băng thông linh hoạt và tùy chỉnh tài nguyên theo từng loại dịch vụ, mang lại hiệu suất vượt trội trong nhiều tình huống sử dụng khác nhau.

2.3.2.4 Ưu điểm và nhược điểm

Công nghệ 4G và 5G mang lại nhiều lợi ích quan trọng trong các hệ thống IoT hiện đại. Với tốc độ truyền dữ liệu cao, 4G cho phép truyền tải video HD, chơi game trực tuyến và vận hành các dịch vụ yêu cầu băng thông rộng mà vẫn duy trì độ trễ thấp. Khi bước sang 5G, tốc độ còn được cải thiện vượt trội cùng với độ trễ cực thấp, giúp đáp ứng tốt các ứng dụng thời gian thực như điều khiển từ xa, xe tự hành hoặc các hệ thống giám sát thông minh. Chất lượng dịch vụ ổn định giúp mạng có thể xử lý đồng thời thoại, video và dữ liệu mà không ảnh hưởng đến hiệu năng. Ngoài ra, 5G còn hỗ trợ mật độ kết nối rất lớn, phù hợp cho các mô hình IoT quy mô cao trong đô thị thông minh hoặc công nghiệp.

Tuy nhiên, cả 4G và 5G đều tồn tại một số hạn chế. Chi phí triển khai hạ tầng cao khiến việc phủ sóng đồng đều gặp khó khăn, đặc biệt tại khu vực nông thôn hoặc vùng sâu vùng xa. Việc xây dựng mạng 5G càng yêu cầu đầu tư lớn hơn do sử dụng nhiều trạm thu phát và băng tần cao. Thiết bị di động sử dụng mạng 4G/5G thường tiêu tốn năng lượng lớn hơn, làm giảm thời gian sử dụng pin. Ngoài ra, dù đã có nhiều cải tiến về bảo mật, các mạng này vẫn đối mặt với rủi ro bị tấn công do kiến trúc phức tạp, đặc biệt ở 5G. Người dùng cũng cần có thiết bị tương thích mới để tận dụng đầy đủ lợi ích của 5G, gây khó khăn cho quá trình phổ cập.

2.3.3 Mạng vệ tinh

2.3.3.1 Giới thiệu

Mạng vệ tinh là công nghệ truyền thông không dây sử dụng các vệ tinh quỹ đạo để truyền dữ liệu giữa các thiết bị mặt đất và trung tâm xử lý. Công nghệ này đặc biệt phù hợp cho các khu vực hẻo lánh, vùng núi, hải đảo hoặc khu vực quân sự nơi hạ tầng viễn thông mặt đất không khả dụng hoặc không ổn định.

Trong các hệ thống giám sát vận chuyển quân sự, mạng vệ tinh đóng vai trò quan trọng như một kênh truyền dự phòng (backup communication), đảm bảo khả năng duy trì kết nối ngay cả khi LoRa hoặc 4G/5G không thể hoạt động.

2.3.3.2 Nguyên lý hoạt động

Mạng vệ tinh hoạt động dựa trên nguyên lý truyền thông vô tuyến giữa thiết bị đầu cuối mặt đất (User Terminal) và các vệ tinh nhân tạo quay quanh Trái Đất. Trong hệ thống này, dữ liệu từ thiết bị IoT được phát đi dưới dạng sóng vô tuyến hướng lên vệ tinh (uplink). Sau khi tiếp nhận, vệ tinh sẽ thực hiện khuếch đại, chuyển tiếp hoặc xử lý sơ bộ tín hiệu, sau đó truyền tín hiệu xuống các trạm mặt đất (downlink) để chuyển tiếp đến trung tâm xử lý dữ liệu hoặc hệ thống đám mây.

Tùy theo quỹ đạo, mạng vệ tinh có thể được phân thành các loại chính như vệ tinh quỹ đạo thấp (LEO – Low Earth Orbit), vệ tinh quỹ đạo trung (MEO – Medium Earth Orbit) và vệ tinh quỹ đạo địa tĩnh (GEO – Geostationary Orbit). Vệ tinh GEO có ưu điểm phủ sóng rộng và ổn định nhưng có độ trễ lớn do khoảng cách xa. Trong khi đó, các hệ thống vệ tinh LEO có độ trễ thấp hơn nhờ quỹ đạo gần Trái Đất, tuy nhiên cần một chùm vệ tinh để đảm bảo phủ sóng liên tục.

Trong các hệ thống IoT giám sát vận chuyển, thiết bị đầu cuối thường được tích hợp module truyền thông vệ tinh chuyên dụng. Khi các kênh truyền mặt đất như LoRa hoặc 4G/5G không khả dụng, thiết bị sẽ tự động chuyển sang chế độ truyền vệ tinh để gửi dữ liệu quan trọng như tọa độ GPS, trạng thái phương tiện hoặc cảnh báo khẩn cấp. Cơ chế này giúp đảm bảo tính liên tục của hệ thống giám sát, đặc biệt trong các môi trường khắc nghiệt hoặc khu vực không có hạ tầng viễn thông.

2.3.3.3 Đặc điểm nổi bật

Mạng vệ tinh có khả năng phủ sóng toàn cầu, không phụ thuộc vào hạ tầng viễn thông mặt đất. Tuy nhiên, do khoảng cách truyền rất xa, độ trễ của mạng vệ tinh thường cao (250–600ms) và tiêu thụ năng lượng lớn hơn so với các công nghệ truyền thông khác. Chi phí triển khai và vận hành cũng cao, bao gồm chi phí thiết bị đầu cuối và thuê kênh truyền vệ tinh.

2.3.3.4 Ưu điểm và nhược điểm

Ưu điểm lớn nhất của mạng vệ tinh là khả năng phủ sóng toàn cầu và tính độc lập hoàn toàn với hạ tầng viễn thông mặt đất. Nhờ đặc điểm này, mạng vệ tinh có thể duy trì kết nối ổn định tại các khu vực vùng sâu vùng xa, hải đảo, sa mạc hoặc khu vực quân sự đặc thù, nơi mà các công nghệ như LoRa hay 4G/5G không thể triển khai hiệu quả. Điều này giúp mạng vệ tinh trở thành một giải

pháp truyền thông đáng tin cậy trong các hệ thống yêu cầu tính sẵn sàng cao và khả năng hoạt động liên tục.

Ngoài ra, mạng vệ tinh thường được sử dụng như một kênh truyền dự phòng (redundant communication) trong các hệ thống quan trọng. Khi kết hợp với các công nghệ truyền thông khác, vệ tinh giúp tăng độ tin cậy tổng thể của hệ thống, giảm nguy cơ mất dữ liệu trong trường hợp xảy ra sự cố hạ tầng hoặc mất kết nối cục bộ. Đối với các ứng dụng giám sát vận chuyển quân sự, khả năng duy trì liên lạc trong mọi điều kiện địa lý và môi trường là một lợi thế mang tính chiến lược.

Tuy nhiên, mạng vệ tinh cũng tồn tại nhiều hạn chế. Do khoảng cách truyền tín hiệu rất lớn, độ trễ của mạng vệ tinh thường cao hơn đáng kể so với các công nghệ truyền thông mặt đất, đặc biệt đối với các hệ thống vệ tinh quỹ đạo địa tĩnh. Độ trễ này có thể ảnh hưởng đến các ứng dụng yêu cầu phản hồi thời gian thực hoặc điều khiển tức thì.

Bên cạnh đó, chi phí triển khai và vận hành mạng vệ tinh tương đối cao, bao gồm chi phí thiết bị đầu cuối, phí thuê băng thông và chi phí duy trì hệ thống vệ tinh. Thiết bị truyền thông vệ tinh cũng tiêu thụ năng lượng lớn hơn, không phù hợp cho các node IoT chạy pin trong thời gian dài. Do những hạn chế này, mạng vệ tinh thường không được sử dụng làm kênh truyền chính mà đóng vai trò bổ trợ, chỉ kích hoạt trong các tình huống đặc biệt khi các phương thức truyền thông khác không khả dụng.

2.4 Các giao thức tầng ứng dụng

2.4.1 MQTT

2.4.1.1 Giới thiệu

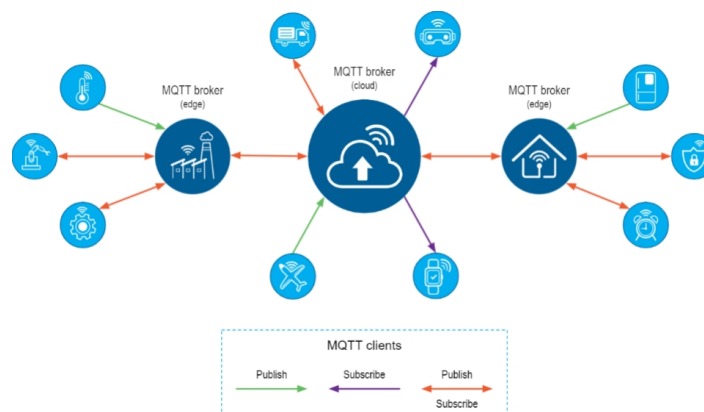
MQTT, viết tắt của Message Queuing Telemetry Transport, là giao thức truyền thông được sử dụng cho các thiết bị IoT với băng thông thấp, độ tin cậy cao và khả năng được sử dụng trong mạng lưới không ổn định.

MQTT là một lựa chọn lý tưởng trong các môi trường mà giá mạng viễn thông đắt đỏ hoặc băng thông thấp hay thiếu tin cậy, hoặc khi chạy trên thiết bị nhúng bị giới hạn về tài nguyên tốc độ và bộ nhớ. Bởi vì giao thức này sử dụng băng thông thấp trong môi trường có độ trễ cao nên nó là một giao thức lý tưởng cho các ứng dụng M2M.

2.4.1.2 Nguyên lý hoạt động

MQTT hoạt động dựa trên mô hình Publish/Subscribe thông qua kiến trúc Client/Server, trong đó Broker đóng vai trò trung gian xử lý toàn bộ quá trình truyền thông. Các thiết bị IoT (Client) có thể gửi dữ liệu dưới dạng thông điệp đến một chủ đề với vai trò Publisher, hoặc đăng ký nhận thông điệp từ các chủ đề đó với vai trò Subscriber. Nhờ cơ chế tách biệt giữa nơi gửi và nơi nhận, MQTT giúp giảm tải băng thông, tối ưu tài nguyên và duy trì hiệu quả hoạt động ngay cả trong môi trường mạng yếu, rất phù hợp với các hệ thống IoT.

Broker được xem như lõi trung tâm của toàn bộ hệ thống, chịu trách nhiệm tiếp nhận thông điệp từ Publisher, xử lý và phân phối chúng đến các Subscriber tương ứng. Bên cạnh nhiệm vụ chính này, Broker còn có thể đảm nhận các chức năng hỗ trợ như quản lý hàng đợi, lưu trữ thông điệp, ghi log hệ thống, xác thực và mã hóa nhằm đảm bảo an toàn trong truyền thông. Chính nhờ sự linh hoạt và cấu trúc nhẹ, MQTT trở thành một trong những giao thức phổ biến nhất trong các ứng dụng IoT hiện nay.



Hình 2.4.1: Minh họa về nguyên lý hoạt động của MQTT

2.4.1.3 Đặc điểm nổi bật

Công nghệ MQTT sở hữu nhiều đặc điểm nổi bật giúp nó trở thành một trong những giao thức tối ưu cho các hệ thống IoT hiện đại. Trước hết, MQTT sử dụng mô hình truyền thông Pub/Sub, cho phép việc trao đổi thông điệp diễn ra theo hướng phân tán và tách biệt hoàn toàn giữa bên gửi và bên nhận. Nhờ đó, việc truyền tải dữ liệu diễn ra ngay lập tức mà không phụ thuộc vào nội dung hay bản chất của thông điệp, giúp giảm độ phức tạp và tài nguyên xử lý trên thiết bị. Giao thức này hoạt động dựa trên nền tảng TCP/IP, đảm bảo độ tin cậy trong truyền dẫn và duy trì kết nối ổn định giữa các thiết bị IoT.

Một trong những ưu điểm quan trọng của MQTT là khả năng hỗ trợ ba mức chất lượng dịch vụ (Quality of Service, hay QoS), cho phép lựa chọn độ tin cậy phù hợp tùy theo ứng dụng. Ở mức QoS 0, thông điệp được gửi đúng một lần nhưng không có cơ chế xác nhận ngoài TCP/IP. Với QoS 1, phía gửi đảm bảo dữ liệu được nhận ít nhất một lần, có thể dẫn đến việc nhận trùng lặp thông điệp. Mức QoS 2 cung cấp độ tin cậy cao nhất khi đảm bảo mỗi thông điệp chỉ được nhận đúng một lần thông qua cơ chế bắt tay bốn bước. Bên cạnh đó, phân bao gói dữ liệu (packet) của MQTT rất nhỏ và được tối ưu đến mức tối thiểu, giúp giảm tải đáng kể cho đường truyền và phù hợp với các mạng băng thông thấp.

2.4.1.4 Ưu điểm và nhược điểm

MQTT mang lại nhiều ưu điểm quan trọng nhờ cơ chế hoạt động nhẹ, linh hoạt và phù hợp với đặc thù của các hệ thống IoT. Với mô hình Pub/Sub và phân bao gói dữ liệu nhỏ, MQTT giúp truyền tải thông tin hiệu quả, giảm đáng kể băng thông tiêu thụ và tối ưu việc sử dụng tài nguyên mạng. Tính chất hoạt động này cũng giúp tăng khả năng mở rộng hệ thống và duy trì hiệu suất tốt trong điều kiện mạng yếu hoặc hạn chế. MQTT rất phù hợp cho các ứng dụng điều khiển, giám sát từ xa như trong SCADA, IoT công nghiệp, hay các hệ thống thu thập dữ liệu cảm biến.

Ngoài ra, giao thức có chi phí vận hành thấp, hỗ trợ bảo mật thông qua các lớp giao thức như TLS/SSL và được tin dùng bởi nhiều tập đoàn lớn. Việc phát triển ứng dụng dựa trên MQTT cũng được đơn giản hóa, giúp rút ngắn thời gian triển khai, đồng thời cơ chế Pub/Sub cho phép thu thập lượng dữ liệu lớn với băng thông ít hơn so với các giao thức truyền thống.

Tuy nhiên, MQTT cũng tồn tại một số hạn chế nhất định. Khi so sánh với CoAP, một giao thức hướng tài nguyên cho IoT, tốc độ truyền của MQTT đôi khi chậm hơn, đặc biệt trong các ứng dụng yêu cầu phản hồi tức thời. CoAP sử dụng mô hình tài nguyên tĩnh, trong khi MQTT hoạt động dựa trên cơ chế đăng ký động, khiến việc quản lý nội dung đôi khi phức tạp hơn.

Một nhược điểm khác là MQTT không tích hợp sẵn cơ chế mã hóa dữ liệu, buộc hệ thống phải sử dụng các giao thức bảo mật bổ sung như TLS/SSL để đảm bảo an toàn. Cuối cùng, khả năng mở rộng ở quy mô rất lớn vẫn là thách thức, do Broker trở thành điểm trung tâm cần xử lý lượng lớn kết nối đồng thời, dễ dẫn đến quá tải nếu không được tối ưu đúng cách.

2.4.2 REST (HTTP)

2.4.2.1 Giới thiệu

REST (Representational State Transfer) là một phong cách kiến trúc phần mềm (architectural style) được sử dụng phổ biến trong việc xây dựng các hệ thống phân tán và dịch vụ web. REST thường được triển khai dựa trên giao thức HTTP, cho phép các hệ thống khác nhau giao tiếp với nhau thông qua các yêu cầu (request) và phản hồi (response) theo mô hình client-server.

Trong các hệ thống IoT hiện đại, RESTful API đóng vai trò quan trọng trong việc kết nối giữa tầng thiết bị, tầng gateway và tầng ứng dụng hoặc đám mây. REST được sử dụng rộng rãi để thu thập dữ liệu cảm biến, quản lý thiết bị, cấu hình hệ thống và cung cấp dữ liệu cho các ứng dụng giám sát, dashboard hoặc hệ thống phân tích. Nhờ tính đơn giản, phổ biến và khả năng tương thích cao, REST trở thành một lựa chọn phù hợp cho các hệ thống IoT có yêu cầu tích hợp với nền tảng web hoặc hệ thống quản lý trung tâm.

2.4.2.2 Nguyên lý hoạt động

REST hoạt động dựa trên mô hình client-server, trong đó client (thiết bị IoT, gateway hoặc ứng dụng) gửi các yêu cầu HTTP đến server để truy cập hoặc thao tác với tài nguyên. Mỗi tài nguyên trong hệ thống được định danh duy nhất bằng một URI (Uniform Resource Identifier) và được biểu diễn dưới các định dạng dữ liệu chuẩn như JSON hoặc XML.

Các thao tác trên tài nguyên được thực hiện thông qua các phương thức HTTP tiêu chuẩn. Phương thức GET được sử dụng để truy vấn dữ liệu, POST dùng để tạo mới tài nguyên, PUT hoặc PATCH để cập nhật dữ liệu, và DELETE để xóa tài nguyên. Server sẽ xử lý yêu cầu và trả về phản hồi tương ứng kèm theo mã trạng thái HTTP (status code) nhằm thông báo kết quả thực hiện.

Trong hệ thống IoT, thiết bị hoặc gateway thường đóng vai trò client, định kỳ gửi dữ liệu cảm biến lên server thông qua các yêu cầu POST hoặc PUT. Server sau đó lưu trữ dữ liệu, xử lý và cung cấp lại thông tin cho các ứng dụng giám sát thông qua các yêu cầu GET. REST không duy trì trạng thái phiên làm việc (stateless), do đó mỗi yêu cầu đều chứa đầy đủ thông tin cần thiết để server xử lý, giúp hệ thống dễ mở rộng và quản lý.

2.4.2.3 Đặc điểm nổi bật

Một trong những đặc điểm nổi bật của REST là tính không trạng thái (stateless), giúp giảm tải cho server và tăng khả năng mở rộng hệ thống. Mỗi yêu cầu HTTP đều độc lập và không phụ thuộc vào các yêu cầu trước đó, phù hợp với các hệ thống phân tán quy mô lớn.

REST tận dụng hạ tầng sẵn có của giao thức HTTP, bao gồm cơ chế định tuyến, mã trạng thái, caching và bảo mật thông qua HTTPS. Việc sử dụng định dạng dữ liệu nhẹ như JSON giúp giảm dung lượng truyền tải và tăng tốc độ xử lý, đặc biệt phù hợp với các ứng dụng web và hệ thống giám sát IoT.

Ngoài ra, REST có khả năng tương thích cao với nhiều nền tảng và ngôn ngữ lập trình khác nhau. Các API RESTful có thể dễ dàng được tích hợp với ứng dụng web, mobile hoặc các hệ thống backend, giúp đơn giản hóa quá trình phát triển và bảo trì hệ thống IoT.

2.4.2.4 Ưu điểm và nhược điểm

REST mang lại nhiều ưu điểm đáng kể trong các hệ thống IoT. Nhờ tính đơn giản và phổ biến của HTTP, việc triển khai REST không đòi hỏi hạ tầng phức tạp và dễ dàng tích hợp với các nền tảng web hiện có. REST phù hợp cho các tác vụ quản lý thiết bị, truy vấn dữ liệu lịch sử, hiển thị dashboard và cung cấp API cho các hệ thống phân tích hoặc điều phối trung tâm. Khả năng mở rộng cao và hỗ trợ tốt cho kiến trúc cloud giúp REST trở thành lựa chọn phổ biến trong các hệ thống IoT quy mô vừa và lớn.

Tuy nhiên, REST cũng tồn tại một số hạn chế khi áp dụng cho IoT. Do hoạt động dựa trên HTTP và mô hình request/response, REST không tối ưu cho các ứng dụng yêu cầu truyền dữ liệu liên tục hoặc thời gian thực. Phần bao gói dữ liệu của HTTP tương đối lớn, dẫn đến tiêu thụ băng thông và năng lượng cao hơn so với các giao thức nhẹ như MQTT hoặc CoAP.

Ngoài ra, REST không hỗ trợ cơ chế push dữ liệu một cách tự nhiên, khiến server khó chủ động gửi dữ liệu xuống thiết bị mà phải thông qua các giải pháp bổ sung như polling hoặc WebSocket. Vì những lý do này, REST thường được sử dụng ở tầng ứng dụng hoặc tầng cloud, trong khi các giao thức nhẹ hơn được ưu tiên cho tầng thiết bị và truyền thông thời gian thực trong hệ thống IoT.

2.4.3 GraphQL

2.4.3.1 Giới thiệu

GraphQL là một ngôn ngữ truy vấn và runtime cho API được phát triển bởi Facebook, nhằm khắc phục những hạn chế của các API truyền thống dựa trên REST. Thay vì server quyết định trước cấu trúc dữ liệu trả về, GraphQL cho phép client chủ động xác định chính xác dữ liệu cần truy vấn, từ đó giảm thiểu dữ liệu dư thừa và tối ưu hiệu quả truyền tải.

Trong các hệ thống IoT hiện đại, GraphQL thường được sử dụng ở tầng ứng dụng hoặc tầng cloud, nơi các dashboard giám sát, hệ thống phân tích hoặc nền tảng quản lý cần truy xuất dữ liệu từ nhiều nguồn khác nhau. Nhờ khả năng gom nhiều truy vấn vào một request duy nhất và cấu trúc dữ liệu linh hoạt, GraphQL đặc biệt phù hợp cho các hệ thống IoT có dữ liệu phức tạp, đa dạng và yêu cầu hiển thị động theo nhu cầu người dùng.

2.4.3.2 Nguyên lý hoạt động

GraphQL hoạt động dựa trên mô hình client-server, trong đó client gửi một truy vấn (query) duy nhất đến server thông qua một endpoint cố định. Truy vấn này mô tả chi tiết cấu trúc dữ liệu mong muốn, bao gồm các trường dữ liệu, mối quan hệ và điều kiện lọc. Server sẽ phân tích truy vấn, ánh xạ đến các nguồn dữ liệu tương ứng và trả về kết quả đúng theo cấu trúc mà client yêu cầu.

Không giống REST, nơi mỗi tài nguyên thường gắn với một endpoint riêng, GraphQL chỉ sử dụng một endpoint duy nhất để xử lý tất cả các truy vấn và thao tác dữ liệu. Các thao tác trong GraphQL được chia thành ba loại chính: Query (truy vấn dữ liệu), Mutation (thay đổi hoặc cập nhật dữ liệu) và Subscription (nhận dữ liệu theo thời gian thực). Cơ chế Subscription cho phép server chủ động gửi dữ liệu cập nhật đến client khi có thay đổi, thường được triển khai thông qua WebSocket.

Trong hệ thống IoT, GraphQL server thường được đặt tại tầng backend hoặc cloud, đóng vai trò tổng hợp dữ liệu từ cơ sở dữ liệu, các dịch vụ microservices hoặc các API nội bộ khác. Các ứng dụng giám sát hoặc dashboard sẽ gửi truy vấn GraphQL để lấy dữ liệu cảm biến, trạng thái thiết bị hoặc lịch sử vận chuyển theo cấu trúc linh hoạt, phù hợp với từng ngữ cảnh hiển thị.

2.4.3.3 Đặc điểm nổi bật

Một trong những đặc điểm nổi bật nhất của GraphQL là khả năng truy vấn chính xác những dữ liệu cần thiết, giúp loại bỏ hiện tượng over-fetching (lấy dư dữ liệu) và under-fetching (lấy thiếu dữ liệu) thường gặp trong REST. Điều này đặc biệt có ý nghĩa trong các hệ thống IoT có nhiều loại dữ liệu và cấu trúc phức tạp.

GraphQL sử dụng schema để định nghĩa rõ ràng các kiểu dữ liệu và mối quan hệ giữa chúng, giúp việc phát triển và bảo trì hệ thống trở nên dễ dàng hơn. Schema đóng vai trò như một hợp đồng giữa client và server, đảm bảo tính nhất quán và giảm thiểu lỗi trong quá trình tích hợp. Ngoài ra, GraphQL hỗ trợ mạnh mẽ cho các công cụ tự động sinh tài liệu và kiểm tra truy vấn.

Bên cạnh đó, GraphQL cho phép gom nhiều truy vấn vào một request duy nhất, giúp giảm số lượng kết nối mạng và tăng hiệu quả truyền tải dữ liệu. Khả năng hỗ trợ real-time thông qua Subscription cũng giúp GraphQL phù hợp với các ứng dụng giám sát trực tuyến hoặc dashboard cần cập nhật dữ liệu liên tục.

2.4.3.4 Ưu điểm và nhược điểm

GraphQL mang lại nhiều ưu điểm trong các hệ thống IoT ở tầng ứng dụng và quản lý dữ liệu. Khả năng truy vấn linh hoạt giúp các ứng dụng giám sát dễ dàng tùy chỉnh dữ liệu hiển thị theo nhu cầu mà không cần thay đổi phía server. Việc giảm thiểu dữ liệu dư thừa cũng góp phần tối ưu băng thông và cải thiện hiệu suất tổng thể, đặc biệt khi truy xuất dữ liệu từ nhiều nguồn khác nhau.

Ngoài ra, GraphQL rất phù hợp cho các hệ thống có giao diện người dùng phức tạp, nhiều dashboard hoặc nhiều loại client khác nhau (web, mobile). Schema rõ ràng giúp việc mở rộng và bảo trì hệ thống trở nên thuận tiện, đồng thời hỗ trợ tốt cho kiến trúc microservices và cloud-native.

Tuy nhiên, GraphQL cũng tồn tại một số hạn chế khi áp dụng cho IoT. Việc triển khai GraphQL server phức tạp hơn so với REST, đòi hỏi tài nguyên xử lý lớn hơn và khả năng kiểm soát truy vấn để tránh các truy vấn quá sâu hoặc quá nặng. Điều này làm GraphQL không phù hợp để triển khai trực tiếp trên các thiết bị IoT hạn chế tài nguyên như vi điều khiển hoặc node cảm biến.

Bên cạnh đó, GraphQL không được tối ưu cho các môi trường băng thông thấp hoặc mạng không ổn định, và thường phải kết hợp với các giao thức khác như MQTT hoặc REST để truyền dữ liệu từ thiết bị lên server. Vì vậy, trong các hệ thống IoT, GraphQL chủ yếu đóng vai trò ở tầng backend và ứng dụng, thay vì thay thế hoàn toàn các giao thức truyền thông nhẹ ở tầng thiết bị.

2.4.4 So sánh và lựa chọn giao thức

Trong hệ thống IoT giám sát và điều phối vận chuyển đạn được, việc lựa chọn giao thức truyền thông và kiến trúc hệ thống phù hợp đóng vai trò then chốt, ảnh hưởng trực tiếp đến độ ổn định, độ trễ, mức tiêu thụ năng lượng và độ an toàn của toàn bộ hệ thống. Hệ thống cần đảm bảo khả năng hoạt động liên tục trong môi trường di chuyển, phạm vi rộng, hạ tầng mạng không ổn định, đồng thời vẫn đáp ứng yêu cầu bảo mật và quản lý tập trung.

2

2.4.4.1 So sánh các giao thức tầng ứng dụng

Ở tầng ứng dụng, ba giao thức phổ biến được xem xét bao gồm MQTT, REST (HTTP) và GraphQL. Mỗi giao thức có đặc điểm riêng và phù hợp với các kịch bản sử dụng khác nhau trong hệ thống IoT.

Giao thức	Độ trễ	Băng thông	Năng lượng tiêu thụ	Bảo mật	Độ tin cậy
MQTT	Thấp	Vừa	Thấp	TLS/SSL	Cao
REST	Trung bình	Vừa – Cao	Trung bình	HTTPS	Cao
GraphQL	Trung bình	Trung bình	Trung bình	HTTPS	Trung bình

Bảng 2.4.1: So sánh các giao thức tầng ứng dụng

MQTT nổi bật nhờ cơ chế Publish/Subscribe, phần bao gói dữ liệu nhỏ và khả năng hoạt động ổn định trong môi trường mạng không đáng tin cậy. Điều này giúp MQTT phù hợp cho việc truyền telemetry, dữ liệu cảm biến và cảnh báo trong các hệ thống IoT thời gian thực. Trong khi đó, REST (HTTP) tuy đơn giản và phổ biến nhưng tiêu thụ băng thông và năng lượng cao hơn, không tối ưu cho truyền dữ liệu liên tục. GraphQL mang lại sự linh hoạt trong truy vấn dữ liệu và phù hợp cho tầng ứng dụng hoặc dashboard, tuy nhiên độ phức tạp triển khai cao và không phù hợp cho thiết bị IoT hạn chế tài nguyên.

Do đó, trong hệ thống đề xuất, MQTT được lựa chọn làm giao thức tầng ứng dụng chính cho việc truyền dữ liệu từ gateway và thiết bị IoT lên server.

2.4.4.2 So sánh các công nghệ truyền thông không dây

Ở tầng truyền thông không dây, các công nghệ được xem xét bao gồm LoRa, 4G/5G và mạng vệ tinh. Việc lựa chọn cần cân nhắc đến phạm vi hoạt động, độ trễ, mức tiêu thụ năng lượng và chi phí triển khai.

Công nghệ	Độ trễ	Băng thông	Năng lượng tiêu thụ	Bảo mật	Độ tin cậy
LoRa	0.3–5s	0.3–50 kbps	Rất thấp	AES-128	Trung bình
4G	30–50ms	Cao	Trung bình	Mạnh	Cao
5G	1–10ms	Rất cao	Trung bình – Cao	Rất mạnh	Cao
Mạng vệ tinh	250–600ms	Trung bình	Cao	Tốt	Cao

Bảng 2.4.2: So sánh các công nghệ truyền thông không dây

LoRa có ưu thế vượt trội về phạm vi truyền xa và mức tiêu thụ năng lượng rất thấp, đặc biệt phù hợp cho các thiết bị IoT chạy pin và hoạt động trong thời gian dài. Mặc dù tốc độ truyền thấp và độ trễ cao hơn so với 4G/5G, LoRa vẫn đáp ứng tốt cho các gói dữ liệu nhỏ như GPS, nhiệt độ và cảnh báo rung/lắc.

Ngược lại, 4G và 5G cung cấp tốc độ cao, độ trễ thấp và phù hợp cho các ứng dụng yêu cầu truyền dữ liệu lớn hoặc thời gian thực. Tuy nhiên, các công nghệ này phụ thuộc mạnh vào hạ tầng viễn thông, tiêu thụ năng lượng lớn hơn và phát sinh chi phí vận hành cao, không phù hợp cho thiết bị IoT chạy pin trong thời gian dài. Mạng vệ tinh có khả năng phủ sóng toàn cầu nhưng chi phí cao và độ trễ lớn, do đó chỉ phù hợp làm kênh truyền dự phòng.

2.4.4.3 So sánh các kiến trúc

Bên cạnh việc lựa chọn giao thức truyền thông, kiến trúc hệ thống cũng đóng vai trò quan trọng trong việc đảm bảo hiệu năng, khả năng mở rộng và độ tin cậy của hệ thống IoT. Trong phạm vi nghiên cứu, các kiến trúc phổ biến được xem xét bao gồm Client–Server truyền thống, P2P/Mesh, Distributed/Microservices và kiến trúc SoC Edge Gateway.

Tiêu chí	Client–Server	P2P / Mesh	Distributed / Microservices	SoC Edge Gateway
Độ trễ	Trung bình - cao	Trung bình	Thấp - trung bình	Thấp
Bảo mật	Trung bình	Cao	Cao	Cao
Mở rộng	Trung bình	Trung bình	Rất cao	Trung bình
Mất mạng	Thấp	Cao	Trung bình - cao	Cao
Chi phí	Trung bình	Trung bình - cao	Cao	Cao
Độ phức tạp	Thấp	Cao	Cao	Trung bình - cao

Bảng 2.4.3: So sánh các kiến trúc hệ thống IoT

Từ Bảng 2.4.3 có thể nhận thấy kiến trúc Client–Server tuy đơn giản và dễ triển khai nhưng phụ thuộc mạnh vào máy chủ trung tâm, dẫn đến độ trễ cao và khả năng hoạt động kém khi mất kết nối mạng. Kiến trúc P2P/Mesh có ưu điểm về khả năng tự tổ chức và duy trì hoạt động khi mạng không ổn định, tuy nhiên việc định tuyến phức tạp và khó kiểm soát bảo mật khiến kiến trúc này không phù hợp cho các hệ thống yêu cầu quản lý tập trung và mức độ an toàn cao.

Kiến trúc Distributed/Microservices thể hiện ưu thế rõ rệt về khả năng mở rộng, tính linh hoạt và phù hợp với các hệ thống cloud-native. Tuy nhiên, chi phí triển khai và vận hành cao cùng với độ phức tạp trong quản lý DevOps khiến kiến trúc này chưa tối ưu khi áp dụng trực tiếp cho các thiết bị IoT hạn chế tài nguyên.

Trong khi đó, kiến trúc SoC Edge Gateway cho phép xử lý dữ liệu cục bộ với độ trễ thấp, khả năng hoạt động độc lập khi mất mạng và mức độ bảo mật cao nhờ tích hợp các cơ chế mã hóa phần cứng. Tuy nhiên, kiến trúc này bị giới hạn bởi tài nguyên phần cứng và khả năng mở rộng hệ thống.

Do đó, để tận dụng ưu điểm và khắc phục hạn chế của từng mô hình, hệ thống được đề xuất triển khai theo kiến trúc Hybrid Cloud - Edge. Trong kiến trúc này, tầng Edge (gateway) đảm nhiệm xử lý thời gian thực, lưu trữ tạm thời và phát hiện bất thường cục bộ, đảm bảo hệ thống vẫn hoạt động ổn định ngay cả khi mất kết nối mạng. Tầng Cloud đóng vai trò quản lý tập trung, lưu trữ dữ liệu dài hạn, phân tích dữ liệu lịch sử và hỗ trợ điều phối từ xa.

Sự kết hợp giữa kiến trúc Hybrid Cloud - Edge cùng với công nghệ truyền

thông LoRa và giao thức MQTT giúp hệ thống đáp ứng đồng thời các yêu cầu về độ trễ thấp, tiêu thụ năng lượng thấp, khả năng mở rộng và độ tin cậy cao, phù hợp với bài toán giám sát và điều phối vận chuyển đạn dược trong môi trường quân sự.

2.4.5 Lựa chọn kiến trúc, giao thức và giải pháp tổng thể

2

Dựa trên các phân tích và so sánh ở trên, hệ thống giám sát và điều phối vận chuyển đạn dược được thiết kế theo kiến trúc Hybrid Cloud - Edge. Trong đó, tầng Edge đảm nhiệm các tác vụ xử lý thời gian thực như thu thập dữ liệu cảm biến, phát hiện bất thường và cảnh báo cục bộ ngay cả khi mất kết nối mạng. Tầng Cloud đóng vai trò lưu trữ tập trung, hiển thị dashboard, phân tích dữ liệu lịch sử và hỗ trợ điều phối từ xa.

Về truyền thông, LoRa được lựa chọn làm công nghệ truyền dẫn chính giữa thiết bị và gateway nhờ khả năng truyền xa, tiêu thụ năng lượng thấp và hoạt động ổn định trong môi trường hạ tầng hạn chế. MQTT được sử dụng làm giao thức tầng ứng dụng để truyền dữ liệu telemetry từ gateway lên server, đảm bảo độ tin cậy và hiệu quả băng thông. Các công nghệ 4G/5G và mạng vệ tinh chỉ được xem xét như các kênh bổ trợ hoặc dự phòng trong những tình huống đặc biệt.

Sự kết hợp giữa LoRa, MQTT và kiến trúc Hybrid Cloud - Edge giúp hệ thống đáp ứng đồng thời các yêu cầu về phạm vi hoạt động rộng, tiêu thụ năng lượng thấp, độ ổn định cao và khả năng quản lý tập trung, phù hợp với đặc thù của bài toán giám sát và điều phối vận chuyển đạn dược trong môi trường quân sự.

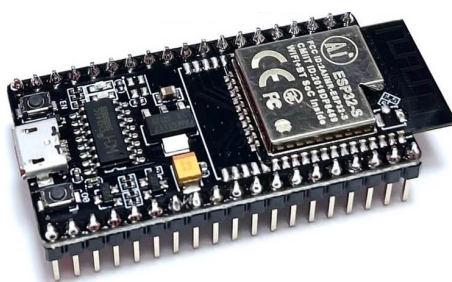
2.5 Các loại cảm biến, thiết bị sử dụng

2.5.1 ESP32 NodeMCU-32S CH340 Ai-Thinker

ESP32 NodeMCU-32S CH340 Ai-Thinker được phát triển trên nền Vi điều khiển trung tâm là ESP32 SoC với công nghệ Wifi, BLE và kiến trúc ARM mới nhất hiện nay, kit có thiết kế phần cứng, firmware và cách sử dụng tương tự Kit NodeMCU ESP8266, với ưu điểm là cách sử dụng dễ dàng, ra chân đầy đủ, tích hợp mạch nạp và giao tiếp UART CH340, thích hợp với các nghiên cứu, ứng dụng về Wifi, BLE, IoT và điều khiển, thu thập dữ liệu qua mạng.

Thông số	Giá trị
SPI Flash	32 Mbits
Dải tần số	2400 - 2483.5 MHz
Bluetooth	BLE 4.2 BR/EDR
WiFi	802.11
Giao diện hỗ trợ	UART/SPI/SDIO/I2C/PWM/I2S/IR/ ADC/DAC
Nguồn sử dụng	5VDC qua cổng Micro USB
Mạch nạp	Tích hợp CH340 UART
Số chân	38 chân cắm 2.54mm, ra đầy đủ chân ESP32
Tích hợp	LED trạng thái, nút nhấn IO0 (BOOT), nút ENABLE
Kích thước	25.4 x 48.3 mm

Bảng 2.5.1: Thông số kỹ thuật của ESP32 NodeMCU-32S CH340 Ai-Thinker



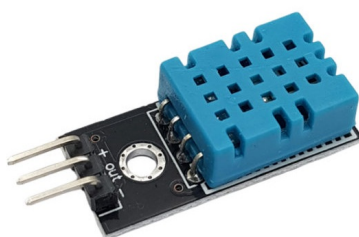
Hình 2.5.1: ESP32 NodeMCU-32S CH340 Ai-Thinker

2.5.2 Cảm biến nhiệt độ, độ ẩm DHT11

DHT11 là cảm biến nhiệt độ, độ ẩm rất thông dụng hiện nay vì chi phí rẻ và rất dễ lấy dữ liệu thông qua giao tiếp 1-wire (giao tiếp digital 1-wire truyền dữ liệu duy nhất). Cảm biến được tích hợp bộ tiền xử lý tín hiệu giúp dữ liệu nhận về được chính xác mà không cần phải qua bất kỳ tính toán nào.

Thông số	Giá trị
Điện áp hoạt động	3-5V
Phạm vi đo độ ẩm	20 - 90% RH
Phạm vi đo nhiệt độ	0 - 50°C
Độ chính xác độ ẩm	±5% RH
Độ chính xác nhiệt độ	±2°C
Tần số lấy mẫu tối đa	1 Hz
Khoảng cách truyền tối đa	20m

Bảng 2.5.2: Thông số kỹ thuật của cảm biến nhiệt độ, độ ẩm DHT11



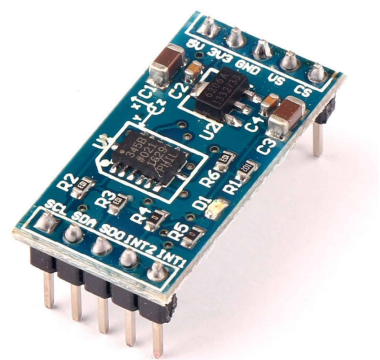
Hình 2.5.2: Cảm biến nhiệt độ, độ ẩm DHT11

2.5.3 Cảm biến gia tốc 3 trục ADXL345

Cảm biến gia tốc 3 trục ADXL345 được dùng để đo gia tốc hoặc độ rung theo ba trục trong hệ tọa độ Descartes. Thiết bị hỗ trợ giao tiếp I2C, dễ tích hợp và có nhiều thư viện mẫu đi kèm. Nhờ độ nhạy cao và kích thước nhỏ gọn, ADXL345 phù hợp với các ứng dụng di động. Cảm biến có thể đo được cả gia tốc tĩnh (dùng để xác định độ nghiêng dựa trên trọng lực) lẫn gia tốc động (phát hiện chuyển động, va đập hoặc rung động).

Thông số	Giá trị
Điện áp hoạt động	3–5VDC
Điện áp giao tiếp	3.3V
Dòng điện tiêu thụ	30μA
Nhiệt độ hoạt động	−40°C - 85°C
Chuẩn giao tiếp	I2C, SPI
Dải đo gia tốc	±2g, ±4g, ±8g, ±16g

Bảng 2.5.3: Thông số kỹ thuật của cảm biến gia tốc 3 trục ADXL345



Hình 2.5.3: Cảm biến gia tốc 3 trục ADXL345

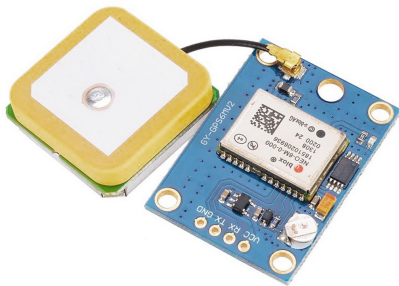
2.5.4 Module GPS NEO-6MV2

Module GPS NEO-6MV2 là một module GPS hoàn chỉnh dựa trên GPS Ublox NEO 6M. Thiết bị này sử dụng công nghệ mới nhất của Ublox để cung cấp thông tin định vị tốt nhất có thể và bao gồm một ăng-ten GPS chủ động tích hợp lớn hơn với chân cắm UART TTL.

Module GPS Ublox có đầu ra TTL nối tiếp, đồng thời có đèn LED hiển thị trạng thái để dễ dàng quan sát trong quá trình sử dụng.

Thông số	Giá trị
Điện áp hoạt động	3–5VDC
Điện áp giao tiếp	3.3V
Kích thước antenna	12 × 12mm
Kích thước module	23 × 30mm
Chuẩn giao tiếp	UART TTL
Baud rate mặc định	9600

Bảng 2.5.4: Thông số kỹ thuật của module GPS NEO-6MV2



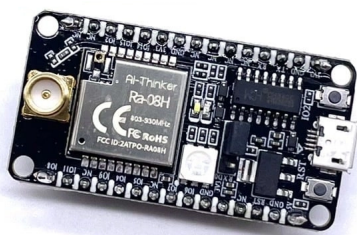
Hình 2.5.4: Module GPS NEO-6MV2

2.5.5 Module truyền thông LoRa Ra-08H Kit

Kit phát triển Ra-08H được sản xuất bởi Ai-Thinker, sử dụng SoC ASR6601 tích hợp lõi vi xử lý 32-bit RISC và bộ thu phát LoRa trong cùng một gói phần cứng. Module hoạt động ở dải tần số cao (868/915 MHz), hỗ trợ chuẩn LoRaWAN cũng như các giao thức truyền thông riêng biệt (P2P). Với ưu điểm về khoảng cách truyền xa và khả năng tiết kiệm năng lượng vượt trội, Ra-08H là lựa chọn tối ưu cho các hệ thống IoT yêu cầu phạm vi phủ sóng rộng. Kit được thiết kế tiện lợi với cổng Micro USB, tích hợp sẵn mạch chuyển đổi USB-UART và ra chân đầy đủ, giúp việc kết nối và lập trình trở nên dễ dàng.

Thông số	Giá trị
SoC trung tâm	ASR6601 (32-bit RISC)
Dải tần số hỗ trợ	803MHz ~ 930MHz (868/915 MHz)
Điện áp hoạt động	5V (Micro USB) hoặc 3.3V (Chân pin)
Giao tiếp ngoại vi	UART, SPI, I2C, GPIO
Công suất phát (Tx)	Tối đa +22dBm
Độ nhạy thu (Rx)	-140 dBm
Bộ nhớ	128 KB Flash, 16 KB SRAM
Antenna	Hỗ trợ đầu nối IPEX

Bảng 2.5.5: Thông số kỹ thuật của Kit LoRa Ra-08H Ai-Thinker



Hình 2.5.5: *Module LoRa Ra-08H Development Board Ai-Thinker*

PHÂN TÍCH YÊU CẦU HỆ THỐNG

3.1 Yêu cầu chức năng

3.1.1 Người dùng

- Có thể theo dõi các thông số quan trọng của quá trình vận chuyển theo thời gian thực trên ứng dụng và web, bao gồm độ rung, nhiệt độ, vị trí GPS và thời gian cập nhật.
- Có thể xem vị trí hiện tại và toàn bộ lộ trình của phương tiện trên bản đồ, đồng thời cập nhật tuyến đường khi cần thiết.
- Có thể nhận cảnh báo ngay lập tức qua ứng dụng hoặc web khi hệ thống phát hiện nhiệt độ hoặc độ rung vượt ngưỡng cho phép hoặc khi phương tiện đi lệch lộ trình.
- Có thể giám sát trạng thái vận chuyển và tiến độ của chuyển hàng.
- Có thể xem và truy xuất lịch sử của các thông số vận chuyển và các sự kiện cảnh báo đã xảy ra.
- Có thể quản lý quyền truy cập và phân quyền cho các vai trò khác nhau như quản trị viên, điều phối viên, lái xe và kỹ thuật viên.
- Quản trị viên có thể xem nhật ký đăng nhập và thao tác của tất cả người dùng trong hệ thống.

3.1.2 Hệ thống

- Phải liên tục thu thập và xử lý dữ liệu từ các cảm biến độ rung, nhiệt độ và thiết bị GPS.
- Phải đồng bộ hóa các dữ liệu này lên hệ thống trung tâm và hiển thị theo thời gian thực trên giao diện người dùng.
- Phải phân tích dữ liệu để đề xuất lộ trình tối ưu dựa trên các thông số đã thu được.
- Phải tự động xác định và kích hoạt cảnh báo khi các thông số vượt ngưỡng hoặc khi vị trí phương tiện đi chệch khỏi lộ trình đã định.
- Phải gửi thông báo cảnh báo đến trung tâm điều hành và người có thẩm quyền qua ứng dụng hoặc web.
- Phải lưu trữ và quản lý toàn bộ lịch sử vận chuyển và dữ liệu liên quan một cách an toàn.
- Phải thực hiện xác thực người dùng trước khi cấp quyền truy cập hệ thống.
- Phải thực hiện xác thực thiết bị trước khi chấp nhận dữ liệu gửi về.
- Phải mã hóa dữ liệu trong quá trình truyền tải và khi lưu trữ để đảm bảo bảo mật dữ liệu.

3

3.2 Yêu cầu phi chức năng

3.2.1 Hiệu năng

- Hệ thống phải đảm bảo truyền dữ liệu từ thiết bị đến máy chủ trong khoảng 2-3 giây.
- Dữ liệu mới phải được cập nhật lên giao diện người dùng trong vòng dưới 5 giây kể từ khi dữ liệu được gửi về.

3.2.2 Tính sẵn sàng

- Hệ thống phải đảm bảo sẵn sàng hoạt động trên 90% khi có kết nối mạng ổn định.
- Dữ liệu phải được thu thập và lưu trữ liên tục khi hệ thống hoạt động.

3.2.3 Khả năng mở rộng

- Hệ thống phải hỗ trợ cùng lúc nhiều thiết bị, với mức tối thiểu là 4 thiết bị đang hoạt động đồng thời.
- Thiết kế hệ thống phải cho phép tích hợp thêm các loại cảm biến mới khác nếu có nhu cầu giám sát mở rộng.

3.2.4 Tính dễ sử dụng

- Giao diện người dùng phải trực quan, dễ nhìn và dễ thao tác.
- Người dùng mới có thể làm quen với các chức năng chính của hệ thống trong không quá 15 phút.

3.2.5 Tính tương thích

- Hệ thống phải tương thích với các thiết bị IoT phổ biến trên thị trường.
- Ứng dụng di động phải hỗ trợ tốt trên nền tảng Android.

3.2.6 Khả năng bảo trì

- Hệ thống phải được thiết kế theo hướng module hóa để cho phép bảo trì hoặc nâng cấp từng phần mà không làm ảnh hưởng đến toàn bộ hoạt động.
- Phải cung cấp đầy đủ tài liệu kỹ thuật và hướng dẫn sử dụng.

3.3 Quy trình vận chuyển đạn dược

Quy trình vận chuyển đạn dược được tổ chức theo một chuỗi bước nghiêm ngặt nhằm đảm bảo an toàn tuyệt đối cho phương tiện, hàng hóa và nhân sự. Hệ thống IoT đóng vai trò hỗ trợ giám sát toàn bộ quá trình, kết hợp với các quy định truyền thống của công tác vận chuyển quân sự để nâng cao mức độ an toàn và khả năng ứng phó sự cố.

Trước khi xuất phát, phương tiện và lô đạn dược được kiểm tra toàn diện về tình trạng kỹ thuật, bao gói, niêm phong và khối lượng theo đúng tiêu chuẩn an toàn. Các cảm biến IoT như nhiệt độ, độ rung và GPS được kích hoạt, kiểm tra tín hiệu và xác thực kết nối với hệ thống trung tâm. Điều phối viên tiến hành cấu hình tuyến đường, thông số giám sát và thiết lập vùng địa lý cho chuyển đi.

Tại điểm xuất phát, đạn dược xếp lên phương tiện theo đúng quy định về an toàn và trọng tải. Sau khi hoàn tất, lực lượng phụ trách thực hiện các bước xác nhận bàn giao cần thiết cho tài xế và ghi nhận thông tin vào hệ thống trước khi khởi hành.

Khi phương tiện bắt đầu di chuyển, thiết bị IoT trên xe sẽ gửi dữ liệu định kỳ về hệ thống trung tâm, bao gồm vị trí GPS, tốc độ, nhiệt độ và độ rung. Hệ thống dựa trên các dữ liệu này để tự động cập nhật trạng thái chuyển đi và hiển thị lộ trình theo thời gian thực, giúp điều phối viên theo dõi và nắm bắt tình hình liên tục.

Trong suốt hành trình, hệ thống IoT có nhiệm vụ giám sát và phát hiện các điều kiện bất thường như nhiệt độ vượt ngưỡng, rung/lắc mạnh, lệch khỏi tuyến đường quy định, dừng quá lâu ngoài khu vực cho phép hoặc mất kết nối truyền thông. Khi xảy ra sự cố, hệ thống sẽ phát cảnh báo ngay lập tức để điều phối viên kịp thời chỉ đạo, có thể yêu cầu dừng kiểm tra, điều chỉnh hướng di chuyển hoặc huy động lực lượng hỗ trợ gần nhất.

Khi xe đến điểm nhận, tài xế và lực lượng tại kho đích tiến hành kiểm tra niêm phong, xác nhận bàn giao và hoàn tất các thủ tục theo quy định. Thông tin về thời gian, vị trí và nhân sự tiếp nhận được hệ thống ghi nhận đầy đủ.

Sau khi bàn giao hoàn tất, toàn bộ dữ liệu liên quan đến chuyển vận chuyển như lộ trình thực tế, thông số giám sát, cảnh báo và các bước xử lý được lưu trữ trên hệ thống. Các dữ liệu này phục vụ công tác đánh giá rủi ro, truy vết và tối ưu hóa các nhiệm vụ vận chuyển trong tương lai.

3.4 Các rủi ro cần giám sát

Trong quá trình vận chuyển đạn dược, tồn tại nhiều rủi ro tiềm ẩn có thể ảnh hưởng trực tiếp đến an toàn con người, phương tiện và hàng hoá. Để giảm thiểu những nguy cơ này, hệ thống IoT được triển khai nhằm giám sát liên tục, phát hiện sớm bất thường và hỗ trợ điều phối xử lý kịp thời.

Một trong những rủi ro quan trọng là nhiệt độ, bởi đạn dược rất nhạy cảm với môi trường nóng, vì khi xe di chuyển qua khu vực nhiệt độ cao hoặc dừng lâu dưới trời nắng, nhiệt độ bên trong thùng chứa có thể vượt quá ngưỡng an toàn, đòi hỏi hệ thống phải cảnh báo ngay lập tức.

Bên cạnh đó, rung và lắc mạnh do đường xấu, phanh gấp hoặc va chạm cũng là yếu tố nguy hiểm, có thể ảnh hưởng đến độ ổn định của đạn, cảm biến rung giúp phát hiện các xung lực bất thường và gửi cảnh báo cho điều phối viên.

Hệ thống cũng cần phát hiện lệch lộ trình, bởi việc phương tiện đi sai đường có thể dẫn đến nguy cơ an ninh, xâm nhập khu vực cấm hoặc rơi vào tình huống bị theo dõi. Ngoài ra, dừng hoặc bất động bất thường là dấu hiệu của sự cố hoặc can thiệp trái phép, vì vậy dữ liệu thời gian dừng được phân tích để đánh giá mức độ nguy hiểm.

Một rủi ro khác là mất tín hiệu hoặc mất kết nối truyền thông, khiến thiết bị

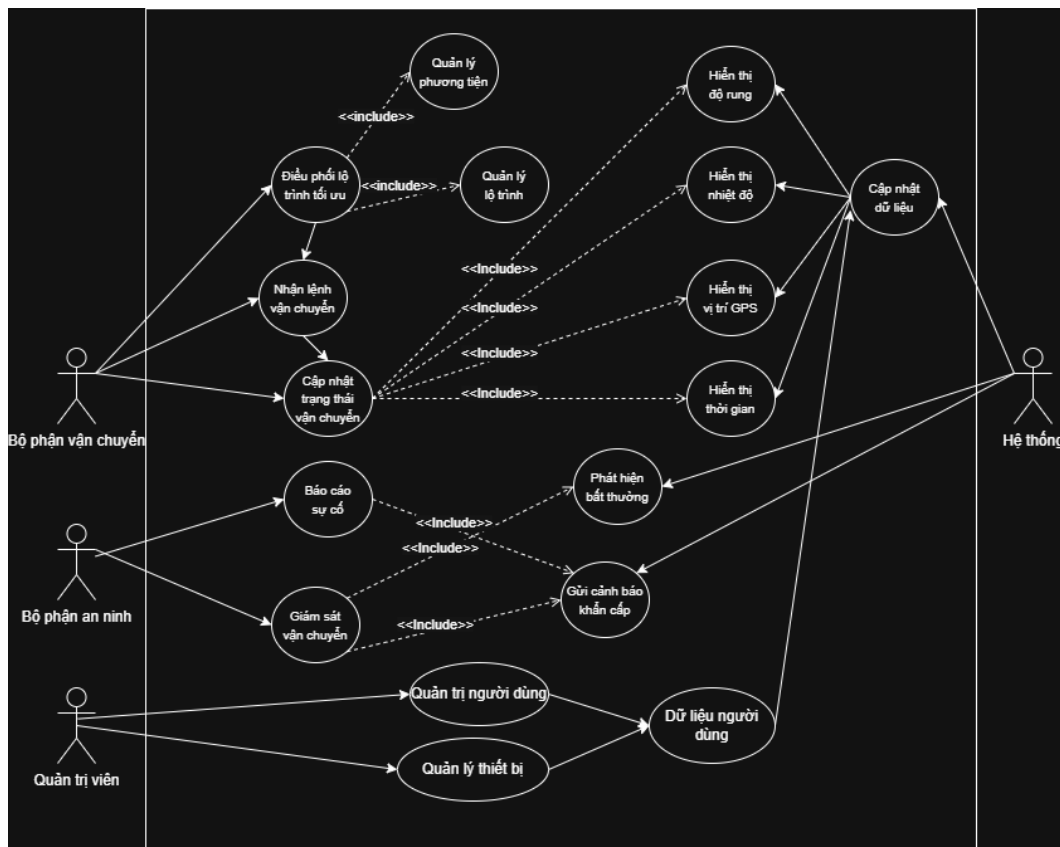
không thể gửi dữ liệu về trung tâm, khi đó, hệ thống phải ghi nhớ dữ liệu và đồng bộ lại khi kết nối được phục hồi. Vấn đề an ninh thiết bị cũng đặc biệt quan trọng, bởi thiết bị IoT có thể bị truy cập trái phép hoặc gửi dữ liệu giả mạo nếu thiếu cơ chế bảo vệ, do đó cần áp dụng mã hóa, ký số và xác thực hai chiều.

Bên cạnh các yếu tố kỹ thuật, hệ thống còn phải tính đến sự cố phương tiện, chẳng hạn như hỏng động cơ, thùng lốp hoặc tai nạn, thường thể hiện qua các tín hiệu rung bất thường, dừng đột ngột hoặc mất tín hiệu. Nguy hiểm nhất là rủi ro cháy nổ, có thể xảy ra khi nhiệt độ tăng nhanh, va chạm mạnh hoặc rò rỉ nhiên liệu; do vậy các cảm biến phải giám sát chặt chẽ để đưa ra cảnh báo sớm.

3

3.5 Use-case

3.5.1 Use-case diagram



Hình 3.5.1: Usecase diagram

3.5.2 Đặc tả use-case

3.5.2.1 Use-case 1: Điều phối lộ trình

Use-case ID	UC02
Use-case Name	Điều phối lộ trình
Actor	Bộ phận kho, Bộ phận vận chuyển
Description	Hệ thống hỗ trợ xây dựng và quản lý lộ trình vận chuyển dựa trên phương tiện, vị trí và trạng thái hàng hóa.
Pre-condition	Hàng hóa và phương tiện đã được khai báo trong hệ thống.
Post-condition	Lộ trình vận chuyển được tạo và phân công cho bộ phận vận chuyển.
Normal flow	1. Người dùng chọn chức năng Điều phối lộ trình. 2. Hệ thống truy xuất thông tin hàng hóa và phương tiện. 3. Người dùng thiết lập hoặc điều chỉnh lộ trình. 4. Hệ thống lưu lộ trình và chuyển sang trạng thái chờ vận chuyển.
Alternative flow	(2a) Thiếu dữ liệu phương tiện: Hệ thống thông báo không thể tạo lộ trình.

Bảng 3.5.1: Đặc tả Use-case UC02: Điều phối lộ trình

3.5.2.2 Use-case 2: Nhận lệnh vận chuyển

Use-case ID	UC03
Use-case Name	Nhận lệnh vận chuyển
Actor	Bộ phận vận chuyển
Description	Nhân viên vận chuyển nhận lệnh và thông tin lộ trình từ hệ thống để bắt đầu quá trình vận chuyển.
Pre-condition	Lộ trình vận chuyển đã được tạo.
Post-condition	Trạng thái vận chuyển được cập nhật sang đang thực hiện.
Normal flow	1. Nhân viên đăng nhập hệ thống. 2. Chọn Nhận lệnh vận chuyển . 3. Hệ thống hiển thị thông tin lộ trình và hàng hóa. 4. Nhân viên xác nhận bắt đầu vận chuyển.
Alternative flow	(4a) Nhân viên từ chối lệnh: Hệ thống ghi nhận và báo về bộ phận kho.

Bảng 3.5.2: Đặc tả Use-case UC03: Nhận lệnh vận chuyển

3.5.2.3 Use-case 3: Cập nhật trạng thái vận chuyển

Use-case ID	UC04
Use-case Name	Cập nhật trạng thái vận chuyển
Actor	Bộ phận vận chuyển, Hệ thống
Description	Trạng thái vận chuyển được cập nhật liên tục dựa trên dữ liệu IoT (GPS, nhiệt độ, độ rung, thời gian).
Pre-condition	Quá trình vận chuyển đang diễn ra.
Post-condition	Trạng thái mới được lưu và hiển thị cho các bộ phận liên quan.
Normal flow	1. Thiết bị IoT gửi dữ liệu về hệ thống. 2. Hệ thống cập nhật trạng thái vận chuyển. 3. Hệ thống hiển thị vị trí GPS, nhiệt độ, độ rung và thời gian.
Alternative flow	(1a) Mất kết nối thiết bị: Hệ thống ghi nhận trạng thái gián đoạn.

Bảng 3.5.3: Đặc tả Use-case UC04: Cập nhật trạng thái vận chuyển

3.5.2.4 Use-case 4: Giám sát vận chuyển

Use-case ID	UC05
Use-case Name	Giám sát vận chuyển
Actor	Bộ phận an ninh
Description	Bộ phận an ninh theo dõi tình trạng vận chuyển theo thời gian thực nhằm phát hiện các dấu hiệu bất thường.
Pre-condition	Dữ liệu vận chuyển đang được cập nhật.
Post-condition	Các bất thường được ghi nhận hoặc cảnh báo.
Normal flow	1. Bộ phận an ninh truy cập chức năng Giám sát vận chuyển . 2. Hệ thống hiển thị trạng thái và dữ liệu cảm biến. 3. Người dùng theo dõi và đánh giá tình trạng vận chuyển.
Alternative flow	(3a) Phát hiện bất thường: Chuyển sang UC06.

3

Bảng 3.5.4: Đặc tả Use-case UC05: Giám sát vận chuyển

3.5.2.5 Use-case 5: Phát hiện bất thường và gửi cảnh báo

Use-case ID	UC06
Use-case Name	Phát hiện bất thường và gửi cảnh báo
Actor	Hệ thống, Bộ phận an ninh
Description	Hệ thống tự động phát hiện các giá trị bất thường và gửi cảnh báo khẩn cấp.
Pre-condition	Có dữ liệu cảm biến từ quá trình vận chuyển.
Post-condition	Cảnh báo được gửi đến bộ phận liên quan.
Normal flow	1. Hệ thống phân tích dữ liệu cảm biến. 2. Phát hiện giá trị vượt ngưỡng cho phép. 3. Hệ thống tạo cảnh báo khẩn cấp. 4. Gửi cảnh báo đến bộ phận an ninh.
Alternative flow	(2a) Dữ liệu nhiễu: Hệ thống bỏ qua và tiếp tục giám sát.

Bảng 3.5.5: Đặc tả Use-case UC06: Phát hiện bất thường và gửi cảnh báo

3.6 Activity diagram

3.6.1 Điều phối lộ trình

Sơ đồ hoạt động này minh họa các khối chức năng của hệ thống, bao gồm ba phần: **Người dùng (Điều phối viên)**, **Hệ thống điều phối**, và **Thiết bị/Phương tiện**.

Người dùng

- Khởi tạo yêu cầu điều phối và nhập ràng buộc: điểm nhận/giao, cửa sổ thời gian, năng lực phương tiện, vùng cấm (geofence), yêu cầu an toàn.
- Duyệt phương án hệ thống đề xuất (KPI/ETA hiển thị trên bản đồ) và phê duyệt (Approve) hoặc quay lại chỉnh ràng buộc nếu chưa phù hợp.

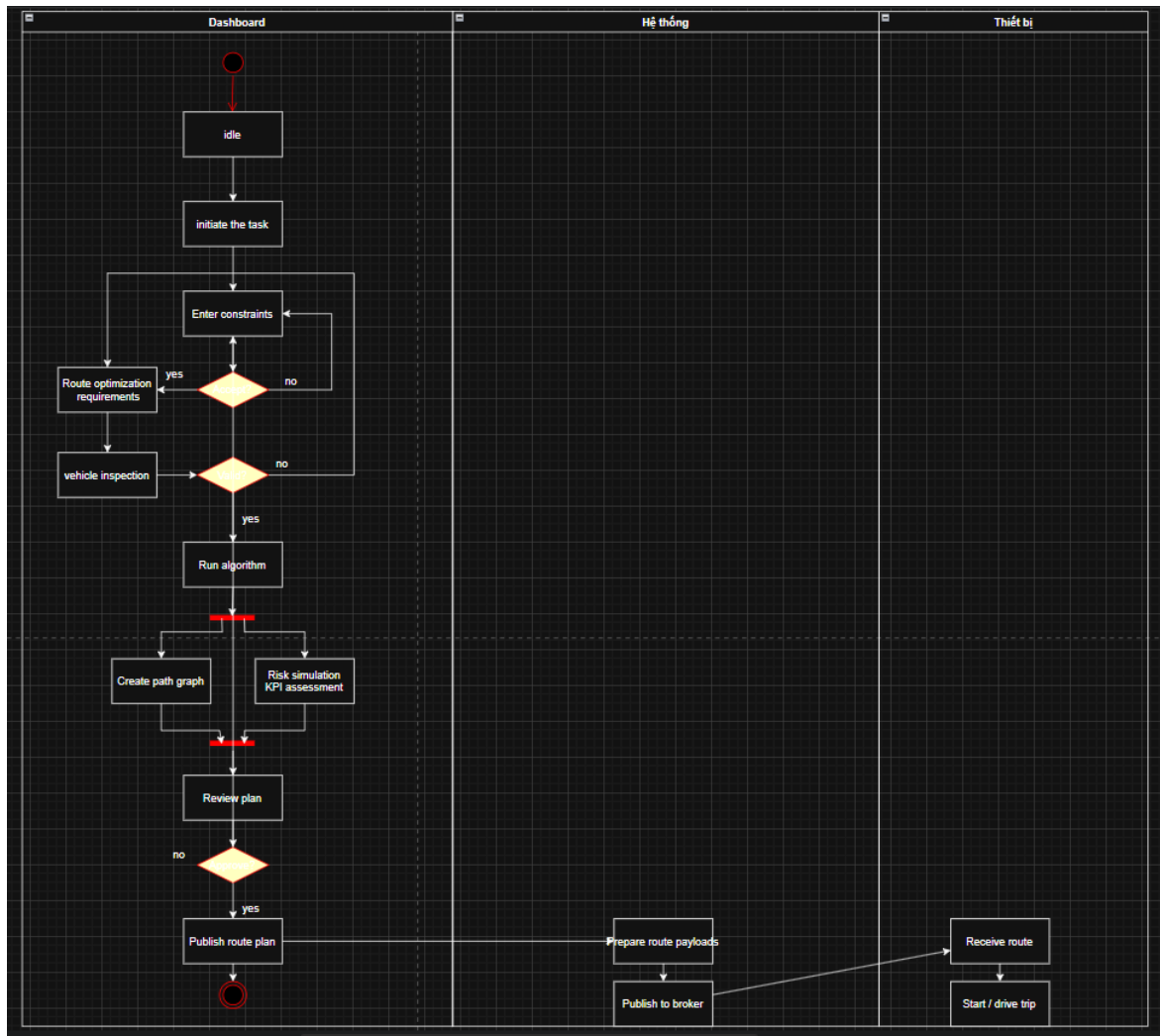
Hệ thống điều phối

- Kiểm tra hợp lệ dữ liệu đầu vào (đủ trường, không trùng, đúng geofence & time window).
- Xây dựng đồ thị lộ trình và chạy thuật toán tối ưu (VRP/TSP mở rộng) để tạo kế hoạch lộ trình kèm KPI/ETA.
- Đóng gói payload (các chặng, tọa độ, ETA, geofence, tham số cảnh báo) và phát hành qua broker.
- Theo dõi ACK từ thiết bị; nếu không nhận ACK thì xếp hàng/Retry theo backoff cho lần gửi tiếp theo.

Thiết bị/Phương tiện

- Nhận lộ trình, gửi ACK xác nhận và bắt đầu hành trình theo kế hoạch đã phát hành.

Kết quả: Kế hoạch lộ trình được tối ưu, phê duyệt và phát hành xuống thiết bị; trạng thái phát hành được theo dõi qua ACK/Retry để đảm bảo thực thi.



Hình 3.6.1: Activity diagram: Điều phối lộ trình

3.6.2 Giám sát & cập nhật trạng thái vận chuyển

Sơ đồ hoạt động này minh họa pipeline thu thập – xác thực – lưu trữ telemetry (GPS, tốc độ, nhiệt, rung), phát hiện bất thường và cập nhật trạng thái chuyển. Quá trình gồm **Thiết bị**, **Hệ thống giám sát** và **Người dùng (Giám sát viên/Điều phối)**.

Thiết bị

- Định kỳ gửi gói dữ liệu. Nếu Online gửi trực tiếp; nếu Offline thì đệm cục bộ và gửi lại khi có kết nối.

Hệ thống giám sát

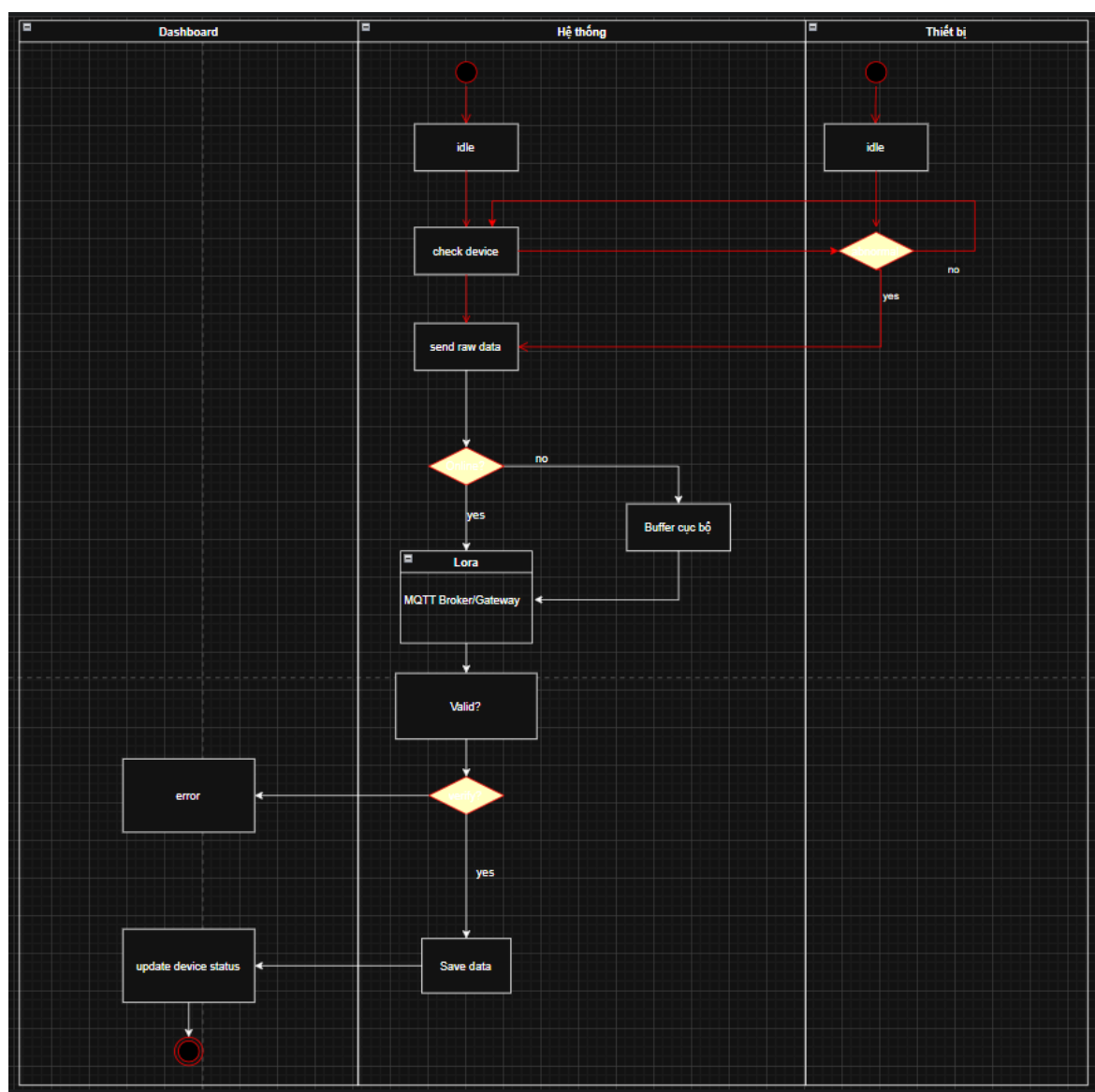
- Nhận gói tin và xác thực (verify): định danh, bảo mật/chữ ký, cấu trúc dữ liệu.

- Nếu không đạt, ghi lỗi và không cập nhật; nếu đạt, lưu dữ liệu và cập nhật trạng thái chuyển (IN_TRANSIT/DELAYED/ALERT/ARRIVED/DELIVERED).
- Đánh giá luật bất thường: vượt ngưỡng an toàn (nhiệt/rung), lệch lộ trình (geofence), dừng bất thường/over-speed. Khi phát hiện ALERT, tạo cảnh báo, có thể mở sự cố và đề xuất hành động; nếu đến đích, xác nhận bàn giao và đóng chuyển.

Người dùng

- Nhận thông báo, xử lý cảnh báo (ack/assign/resolve), và khi cần có thể bổ sung ghi chú để hoàn tất hồ sơ sự cố.

Kết quả: Trạng thái vận chuyển được cập nhật liên tục; bất thường được phát hiện sớm & cảnh báo; dữ liệu được xác thực – lưu trữ an toàn và liên thông với điều phối để tái tối ưu khi cần.



Hình 3.6.2: Activity diagram: Giám sát & cập nhật trạng thái vận chuyển

3.7 Sequence diagram

Phần này trình bày hai sơ đồ trình tự tương ứng với ba khối chức năng cốt lõi của hệ thống: **Điều phối lộ trình** và **Giám sát & cập nhật trạng thái vận chuyển**. Mỗi sơ đồ mô tả chi tiết chuỗi tương tác theo thời gian giữa ba thành phần chính: *Dashboard* (giao diện người dùng), *Hệ thống* (xử lý nghiệp vụ) và *Thiết bị* (thiết bị/ứng dụng triển khai tại hiện trường). Thông qua các sơ đồ này, quá trình xử lý tác vụ, phản hồi và đồng bộ dữ liệu giữa các thành phần được biểu diễn một cách rõ ràng, hỗ trợ cho việc thiết kế, hiện thực và đánh giá hệ thống.

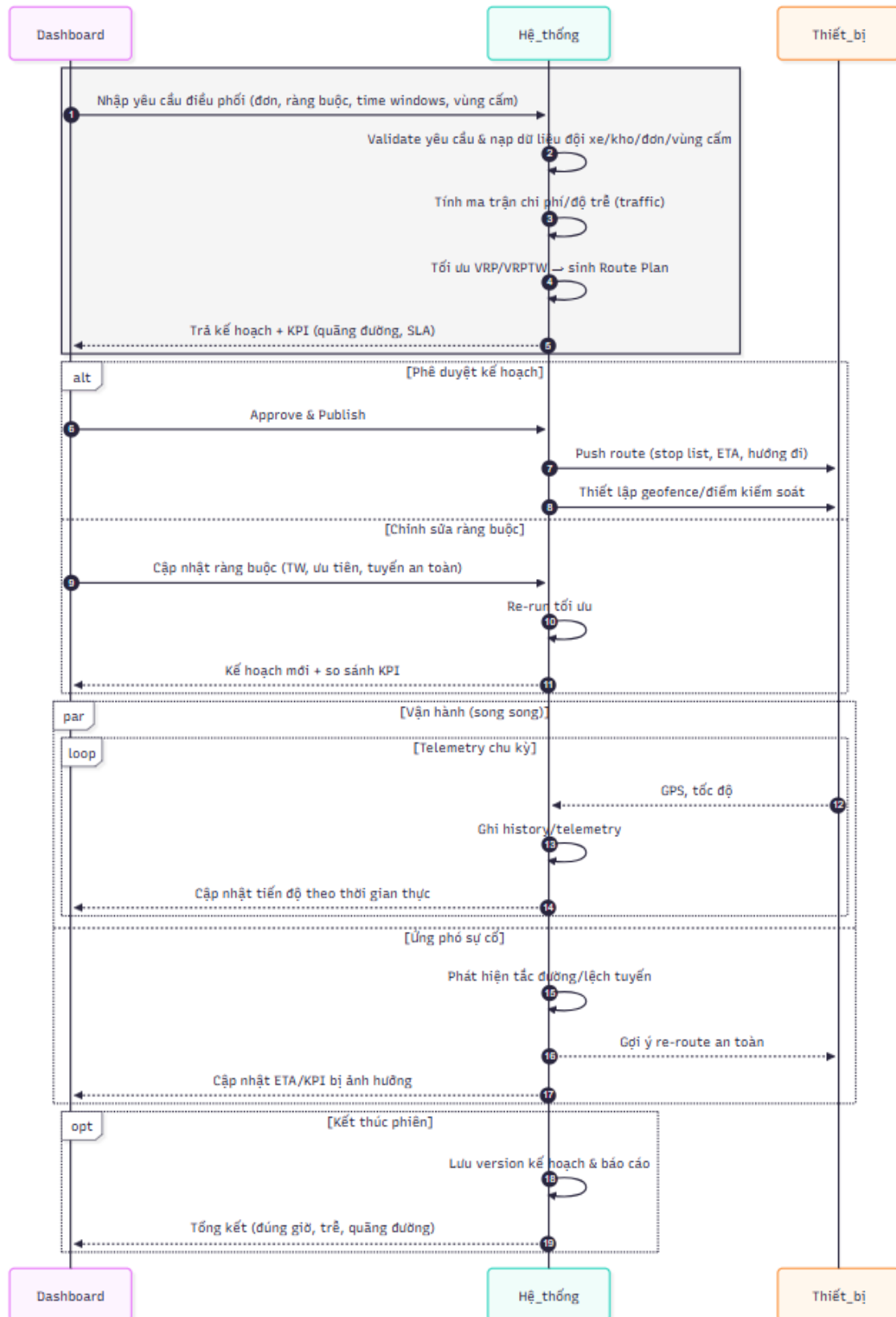
3.7.1 Điều phối lộ trình

Trong chức năng điều phối lộ trình, Dashboard đóng vai trò khởi tạo yêu cầu điều phối bao gồm tập đơn hàng, các ràng buộc vận hành, khoảng thời gian giao (*time windows*) và các khu vực hạn chế. Hệ thống tiếp nhận yêu cầu, kiểm tra tính hợp lệ, tải dữ liệu đội xe, kho và đơn hàng, đồng thời xây dựng ma trận chi phí hoặc thời gian. Dựa trên các dữ liệu này, hệ thống giải bài toán tối ưu (VRP/VRPTW) để sinh ra kế hoạch lộ trình cùng các chỉ số đánh giá (KPI), sau đó gửi kết quả về Dashboard để người vận hành xem trước.

3

Quá trình xử lý phân nhánh thành hai hướng chính. Trường hợp kế hoạch được phê duyệt, hệ thống tiến hành công bố lộ trình và chuyển danh sách điểm dừng, thời gian dự kiến (ETA) đến Thiết bị, đồng thời tạo *geofence* và các điểm kiểm soát dọc tuyến. Ngược lại, nếu người vận hành thực hiện chỉnh sửa, Dashboard gửi lại ràng buộc cập nhật, hệ thống chạy lại thuật toán tối ưu và trả về kế hoạch mới kèm theo so sánh KPI giữa hai phiên bản.

Khi vận hành thực tế diễn ra, Hệ thống và Thiết bị hoạt động song song. Thiết bị gửi dữ liệu *telemetry* định kỳ để hệ thống ghi lại lịch sử, cập nhật tiến độ và tính toán ETA theo thời gian thực, sau đó hiển thị trên Dashboard. Đồng thời, hệ thống liên tục đánh giá điều kiện giao thông và phát hiện lệch tuyến để đưa ra gợi ý điều hướng lại khi cần thiết.

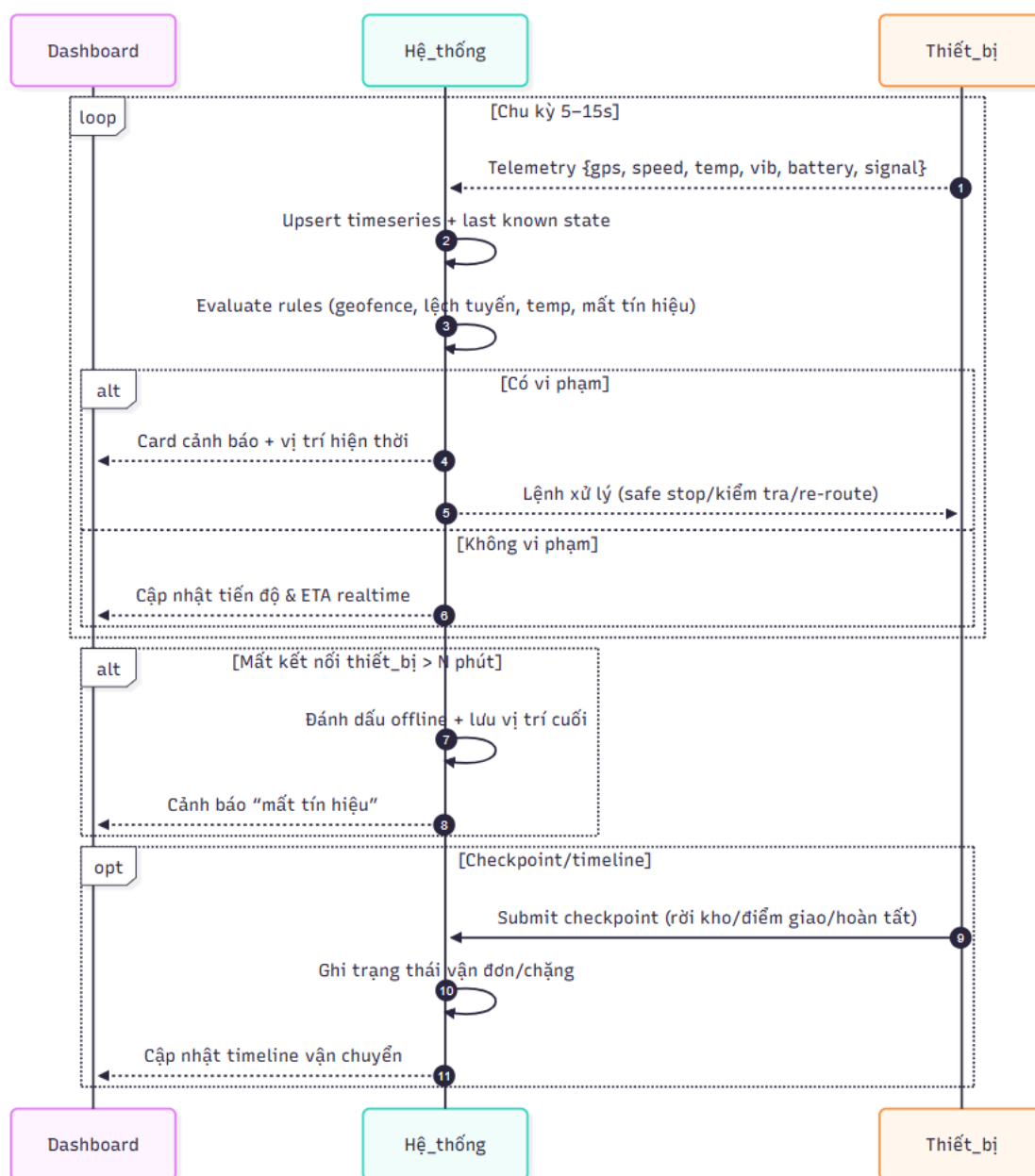


Hình 3.7.1: Sequence diagram: Điều phối lộ trình

3.7.2 Giám sát & cập nhật trạng thái vận chuyển

Chức năng giám sát và cập nhật trạng thái vận chuyển dựa trên luồng dữ liệu *telemetry* do Thiết bị gửi định kỳ trong khoảng 5–15 giây. Dữ liệu bao gồm vị trí GPS, tốc độ, nhiệt độ, mức rung, dung lượng pin và chất lượng tín hiệu. Hệ thống xử lý dữ liệu theo cơ chế *upsert*, đánh giá theo các luật giám sát như geofence, lệch tuyến, vi phạm nhiệt độ hoặc mất tín hiệu. Trên cơ sở đó, luồng xử lý được phân thành hai trường hợp: nếu phát hiện vi phạm, Hệ thống tạo thẻ cảnh báo (kèm vị trí hiện tại), hiển thị trên Dashboard và gửi lệnh xử lý xuống thiết bị như dừng an toàn, kiểm tra hoặc điều hướng lại tuyến; nếu không có vi phạm, hệ thống tiếp tục cập nhật tiến độ và ETA theo thời gian thực.

Trong trường hợp thiết bị mất kết nối, hệ thống phát hiện dựa trên thời gian im lặng vượt ngưỡng, đánh dấu thiết bị ở trạng thái *offline* và thông báo vị trí cuối cùng cho Dashboard để người vận hành triển khai xác minh. Bên cạnh đó, hệ thống hỗ trợ cơ chế checkpoint/timeline, trong đó thiết bị gửi các mốc quan trọng như rời kho, đến điểm giao hoặc hoàn tất chuyến, giúp xây dựng dòng thời gian vận chuyển phục vụ truy vết và đánh giá hiệu suất.

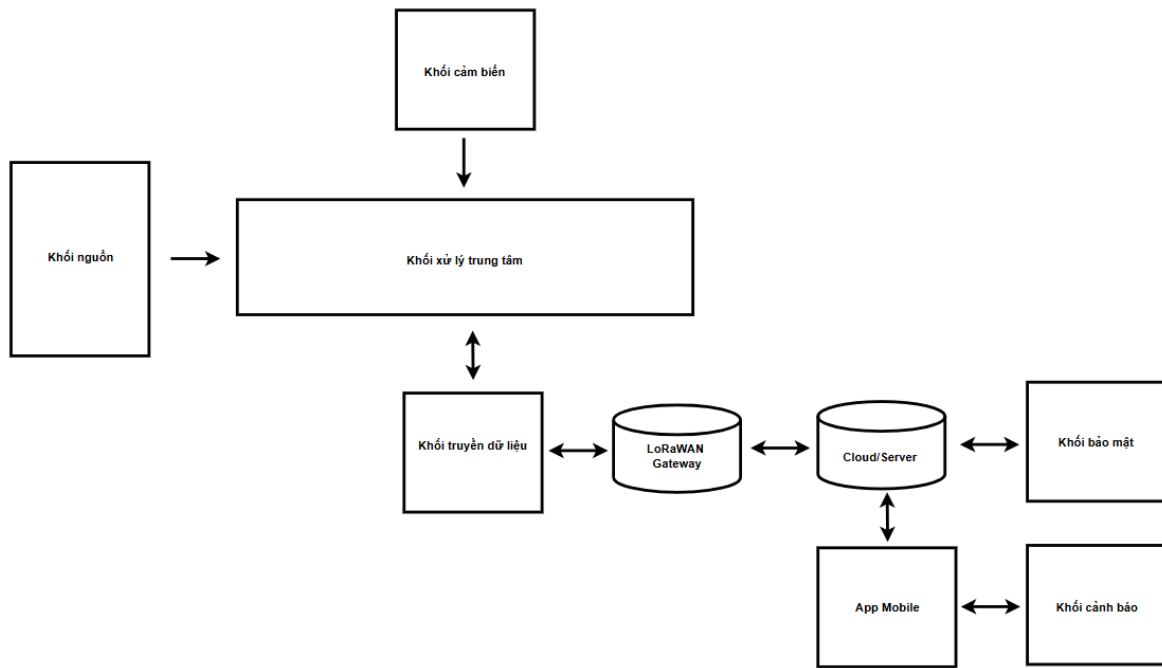


Hình 3.7.2: Sequence diagram: Giám sát & cập nhật trạng thái vận chuyển

THIẾT KẾ HỆ THỐNG

4.1 Sơ đồ kiến trúc tổng thể

Sơ đồ khối chức năng của hệ thống được thể hiện trong Hình 4.1.1. Trung tâm của hệ thống là khối xử lý trung tâm, chịu trách nhiệm thu nhận dữ liệu từ các cảm biến rung, nhiệt độ và GPS, thực hiện tiền xử lý, mã hóa và quản lý luồng thông tin. Khối nguồn đảm bảo cung cấp điện áp ổn định cho toàn bộ hệ thống. Sau khi xử lý, dữ liệu được hiển thị cục bộ trên màn hình và đồng thời truyền qua LoRa/LoRaWAN đến Gateway trước khi chuyển tiếp lên Cloud/Server. Tại đây, dữ liệu được lưu trữ trong cơ sở dữ liệu, quản lý bởi MQTT Broker và tích hợp với khối bảo mật cùng ứng dụng di động để hỗ trợ giám sát, cảnh báo và ra quyết định từ xa.



Hình 4.1.1: Sơ đồ khối của toàn bộ hệ thống

Toàn bộ hệ thống được cấp điện bởi khối nguồn, đảm bảo cung cấp mức điện áp 5V ổn định cho các khối cảm biến, xử lý, hiển thị và truyền thông. Việc duy trì nguồn điện liên tục và ổn định là yếu tố then chốt để đảm bảo thiết bị có thể giám sát hành trình vận chuyển đạn dược mà không bị gián đoạn.

Khối cảm biến đóng vai trò thu thập dữ liệu môi trường và trạng thái chuyển động của phương tiện. Cảm biến rung giám sát các dao động bất thường trong quá trình vận chuyển, cảm biến DHT20 theo dõi nhiệt độ và độ ẩm nhằm đảm bảo điều kiện bảo quản đạn dược luôn nằm trong ngưỡng an toàn, module GPS cung cấp thông tin vị trí địa lý theo thời gian thực, giúp hệ thống giám sát chính xác lộ trình và phát hiện các sai lệch hướng di chuyển.

Dữ liệu sau khi được xử lý tại thiết bị sẽ được truyền đi thông qua khối truyền dữ liệu LoRa/LoRaWAN. Tín hiệu từ các node cảm biến được chuyển đến LoRaWAN Gateway, đóng vai trò cầu nối giữa thiết bị tại hiện trường và không gian lưu trữ trên Cloud. Gateway hỗ trợ quản lý nhiều thiết bị cùng lúc, mã hóa dữ liệu nhận được và chuyển tiếp lên nền tảng server qua giao thức bảo mật MQTT.

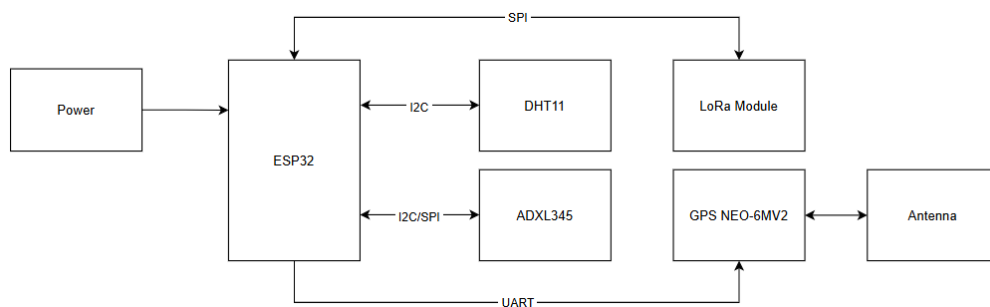
Tại tầng xử lý dữ liệu trung tâm, Cloud/Server đảm nhiệm vai trò tiếp nhận dữ liệu từ Gateway, xử lý, phân loại và lưu trữ trong cơ sở dữ liệu. MQTT Broker được sử dụng để đảm bảo việc truyền nhận dữ liệu giữa các thiết bị và ứng dụng giám sát diễn ra đồng bộ, ổn định theo thời gian thực. Tại đây, người quản lý có thể xem lại lịch sử hành trình, phân tích dữ liệu hoặc đánh giá các chỉ số an

toàn trong suốt quá trình vận chuyển.

Xuyên suốt toàn bộ hệ thống là khối bảo mật, tích hợp ở từng tầng từ cảm biến, Gateway cho tới Cloud. Các cơ chế mã hóa dữ liệu, xác thực thiết bị và kiểm soát quyền truy cập giúp đảm bảo rằng mọi thông tin liên quan đến vận chuyển đạn được đều được bảo vệ, tránh nguy cơ truy cập trái phép hoặc rò rỉ dữ liệu.

Cuối cùng, ứng dụng di động đóng vai trò giao diện tương tác với người quản lý. Ứng dụng hiển thị trạng thái vận chuyển theo thời gian thực, cung cấp bản đồ số để theo dõi lộ trình và nhận cảnh báo tức thời khi hệ thống ghi nhận bất kỳ sự cố nào. Nhờ đó, người quản lý có thể ra quyết định nhanh chóng và điều phối phương tiện hiệu quả hơn.

4.2 Sơ đồ kết nối phần cứng



Hình 4.2.1: Sơ đồ kết nối phần cứng của hệ thống

Sơ đồ kết nối phần cứng minh họa trong Hình 4.2.1 thể hiện kiến trúc tổng thể của các thành phần điện tử trong hệ thống giám sát vận chuyển đạn được. Trung tâm của toàn bộ cấu trúc là ESP32, đóng vai trò vi điều khiển chính, chịu trách nhiệm thu thập dữ liệu từ các cảm biến, truyền thông với các module ngoại vi và gửi dữ liệu đến backend thông qua LoRa. ESP32 đồng thời được cấp nguồn trực tiếp từ khối Power, đảm bảo toàn bộ hệ thống hoạt động ổn định và liên tục.

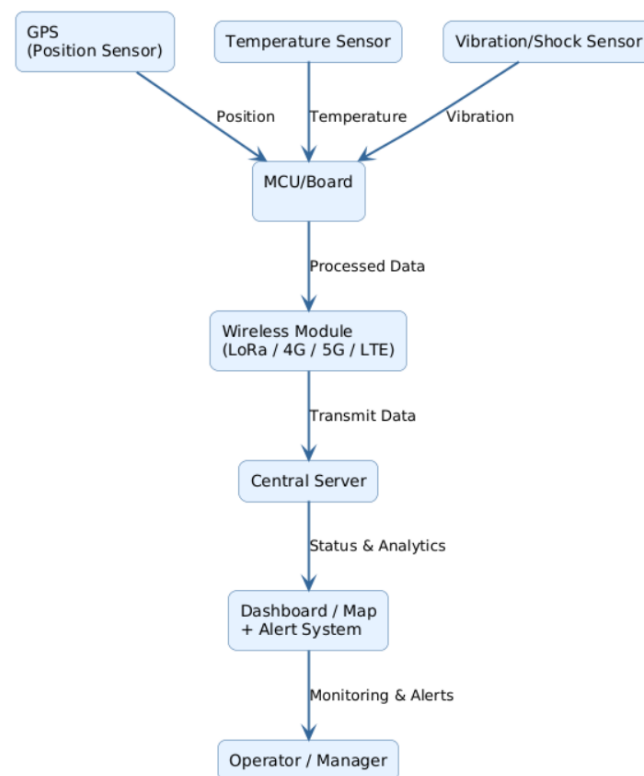
Các cảm biến môi trường và trạng thái được kết nối trực tiếp đến ESP32 thông qua các giao thức phù hợp. Cảm biến DHT11 sử dụng giao tiếp GPIO, cho phép truyền dữ liệu nhiệt độ và độ ẩm theo chu kỳ về vi điều khiển. Tương tự, cảm biến gia tốc ADXL345 được kết nối qua giao tiếp I2C, hỗ trợ ESP32 thu thập thông tin về rung/lắc và chuyển động của phương tiện trong quá trình vận chuyển. Các kết nối này giúp hệ thống ghi nhận đầy đủ các yếu tố ảnh hưởng đến an toàn của lô đạn được.

Đối với việc xác định vị trí, hệ thống sử dụng module GPS NEO-6MV2, kết nối với ESP32 thông qua giao tiếp UART. Module GPS đảm nhiệm việc thu thập tọa độ thời gian thực và truyền về vi điều khiển. Dữ liệu định vị sau đó được ESP32 xử lý và gửi đi, đảm bảo phương tiện luôn được giám sát chặt chẽ trên bản đồ. Module GPS cũng được kết nối với antenna ngoài để cải thiện khả năng thu tín hiệu trong điều kiện môi trường phức tạp.

Để truyền dữ liệu về trung tâm, ESP32 sử dụng module LoRa, kết nối thông qua chuẩn UART. Module LoRa đảm nhiệm việc gửi dữ liệu tầm xa, năng lượng thấp, phù hợp với đặc thù vận chuyển quân sự. Toàn bộ thông tin bao gồm nhiệt độ, độ ẩm, rung/lắc và vị trí GPS được ESP32 tổng hợp và gửi sang LoRa Module, sau đó truyền về Gateway của hệ thống.

4.3 Thiết kế truyền thông dữ liệu

4



Hình 4.3.1: Mô phỏng luồng dữ liệu tổng quan của hệ thống

Hệ thống truyền thông dữ liệu được thiết kế nhằm đảm bảo rằng mọi thông tin thu thập từ phương tiện vận chuyển đều được truyền về trung tâm một cách ổn định, liên tục và an toàn. Mô phỏng luồng dữ liệu tổng quan được thể hiện trong Hình 4.3.1, mô tả đầy đủ hành trình mà dữ liệu đi qua từ khi được tạo ra tại lớp cảm biến đến lúc được trình bày trên bảng điều khiển cho người quản lý.

Ở lớp thấp nhất, các cảm biến đóng vai trò là nguồn tạo dữ liệu. Ba nhóm cảm biến chính gồm GPS, cảm biến nhiệt độ, độ ẩm và cảm biến rung/lắc, liên tục ghi nhận các thông số liên quan đến vị trí phương tiện, điều kiện môi trường và trạng thái chuyển động. Các dữ liệu này mang tính thô và cần được xử lý thêm trước khi truyền đi.

Toàn bộ dữ liệu thô được chuyển tới khối xử lý cục bộ (MCU/Board), nơi thực hiện các tác vụ như lọc nhiễu tín hiệu, định dạng dữ liệu và kiểm tra tính hợp lệ. Việc xử lý sơ bộ giúp giảm tải cho hệ thống trung tâm, đồng thời đảm bảo dữ liệu truyền đi có cấu trúc rõ ràng, giảm rủi ro sai lệch hoặc nhiễu trong quá trình truyền.

Dữ liệu sau khi được chuẩn hóa sẽ được chuyển sang module truyền thông, với hai công nghệ chính là LoRa và 4G/5G. LoRa đóng vai trò nền tảng truyền thông chính của hệ thống nhờ ưu thế về tầm xa, độ ổn định và mức tiêu thụ năng lượng thấp, rất phù hợp với bối cảnh vận chuyển quân sự. Trong khi đó, 4G/5G được xem như kênh bổ trợ giúp duy trì liên lạc trong trường hợp thiết bị đi vào khu vực không phủ sóng LoRa hoặc môi trường truyền dẫn quá nhiễu.

Tại trung tâm, dữ liệu được tiếp nhận bởi máy chủ. Máy chủ có nhiệm vụ lưu trữ, xử lý sâu hơn và phân tích dữ liệu theo các thuật toán phát hiện bất thường. Từ đây, các thông tin quan trọng như cảnh báo, thống kê hay trạng thái vận chuyển sẽ được cập nhật liên tục lên hệ thống giao diện người dùng.

Cuối chuỗi truyền thông là dashboard giám sát, nơi dữ liệu được biểu diễn dưới dạng bản đồ hành trình, biểu đồ nhiệt độ, đồ thị rung/lắc và các cảnh báo theo thời gian thực. Dashboard đóng vai trò then chốt, giúp người quản lý dễ dàng theo dõi toàn bộ quá trình vận chuyển và ra quyết định nhanh chóng khi phát hiện tình huống bất thường.

Thông qua thiết kế truyền thông dữ liệu này, toàn bộ quá trình từ thu thập, xử lý, truyền tải đến hiển thị được đảm bảo tính liên tục, đáng tin cậy và hiệu quả. Đây là nền tảng quan trọng giúp hệ thống vận hành ổn định trong thực tế, đặc biệt trong các tình huống yêu cầu an toàn cao như vận chuyển đạn dược.

4.4 Thiết kế ứng dụng giám sát

Để hỗ trợ việc xây dựng và triển khai ứng dụng giám sát, hệ thống được tổ chức theo kiến trúc tách biệt giữa Backend (xử lý thiết bị IoT) và Frontend (ứng dụng giám sát). Cách tổ chức repository theo hướng Implementation View nhằm đảm bảo việc phát triển, mở rộng và bảo trì được thuận lợi.

4.4.1 Backend

Backend chịu trách nhiệm thu thập dữ liệu từ các cảm biến, thực hiện mã hoá và truyền dữ liệu, đồng thời giao tiếp với hệ thống giám sát trung tâm. Thành phần này đóng vai trò là nguồn cung cấp dữ liệu cốt lõi cho ứng dụng theo dõi trạng thái vận chuyển trong toàn bộ hệ thống.

Cấu trúc thư mục của Backend được tổ chức rõ ràng nhằm hỗ trợ quá trình phát triển, kiểm thử và triển khai trên vi điều khiển ESP32. Các thành phần chính bao gồm mã nguồn, tài liệu và bộ kiểm thử, được sắp xếp theo từng thư mục chức năng như dưới đây:

- `src/` - Chứa toàn bộ mã nguồn chạy trên ESP32
 - `main.c` - Vòng lặp chính của chương trình.
 - `sensors.c/.h` - Thu thập dữ liệu từ GPS, nhiệt độ, độ rung/lắc.
 - `comm_mqtt.c/.h, comm_coap.c/.h` - Giao tiếp bằng giao thức MQTT/CoAP.
 - `lora_driver.c/.h` - Truyền dữ liệu qua mạng LoRa.
 - `security.c/.h` - Thực hiện mã hoá và xác thực thiết bị.
 - `utils.c/.h` - Các hàm tiện ích dùng chung.
- `tests/` - Chứa các kịch bản kiểm thử cảm biến, truyền thông và bảo mật.
- `docs/` - Lưu trữ sơ đồ mạch, tài liệu kiến trúc và hướng dẫn triển khai Backend.

4.4.2 Frontend

Frontend được phát triển bằng Flutter, đóng vai trò cung cấp giao diện trực quan để giám sát trạng thái vận chuyển theo thời gian thực. Ứng dụng hỗ trợ hiển thị bản đồ, cảnh báo, lịch sử dữ liệu và tình trạng của thiết bị trên nền tảng di động hoặc web. Toàn bộ cấu trúc mã nguồn được tổ chức theo mô hình rõ ràng giúp quá trình phát triển, mở rộng và bảo trì trở nên thuận tiện hơn.

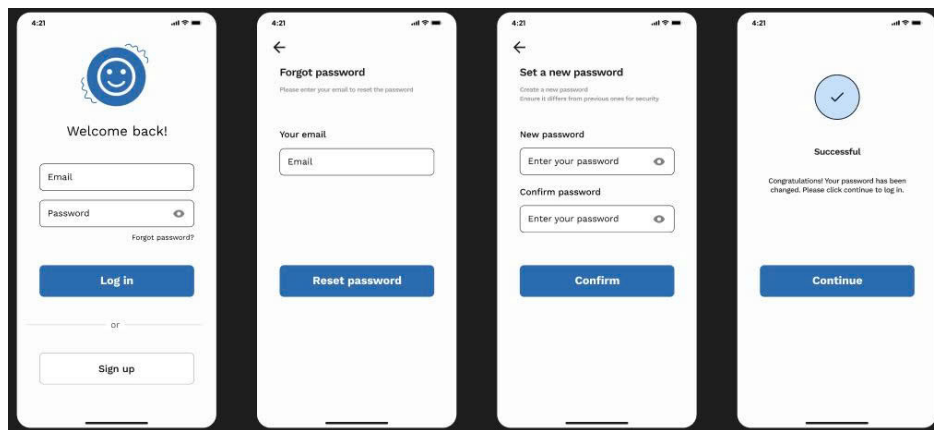
Cấu trúc thư mục của Frontend bao gồm các thành phần chính như sau:

- `lib/` – Thư mục mã nguồn chính của ứng dụng Flutter
 - `main.dart` – Điểm khởi động của ứng dụng.
 - `screens/` – Chứa các màn hình chức năng như Dashboard, MapScreen, AlertScreen và HistoryScreen.
 - `widgets/` – Các thành phần giao diện được tái sử dụng trong nhiều màn hình.

- services/ – Thực hiện kết nối API và xử lý dữ liệu nhận từ Backend.
- models/ – Định nghĩa các mô hình dữ liệu như *Sensor*, *Alert*, *History*.
- utils/ – Chứa các hàm tiện ích và cấu hình chung của ứng dụng.
- assets/ – Lưu trữ hình ảnh, biểu tượng và các tệp cấu hình.
- docs/ – Bao gồm tài liệu thiết kế giao diện, tài liệu kiến trúc Frontend và hướng dẫn sử dụng ứng dụng.

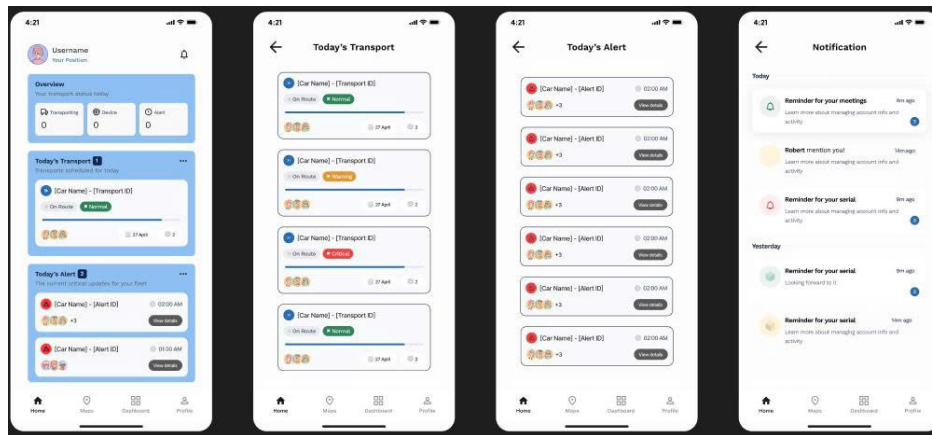
4.5 Thiết kế giao diện người dùng

Hình 4.5.1 mô tả các giao diện liên quan đến chức năng đăng nhập và đăng ký. Người dùng có thể tạo tài khoản mới hoặc đăng nhập vào hệ thống để bắt đầu sử dụng các tính năng giám sát vận chuyển đạn dược.



Hình 4.5.1: Các giao diện liên quan đến phần đăng nhập, đăng ký

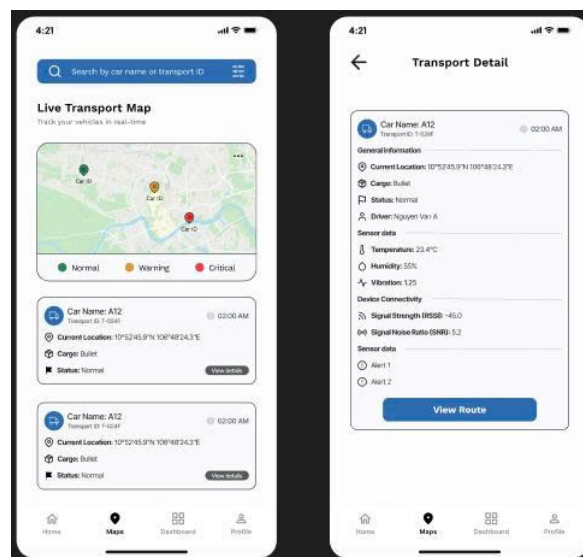
Tiếp theo, Hình 4.5.2 thể hiện giao diện trang Home, nơi tổng hợp các thông tin quan trọng ở dạng tóm tắt. Trang này đóng vai trò trung tâm điều hướng, cung cấp các mục chính như tổng quan, lịch trình vận chuyển và cảnh báo trong ngày.



Hình 4.5.2: Các giao diện liên quan đến trang *Home*

Hình 4.5.3 minh họa giao diện trang Maps, cho phép theo dõi vị trí các phương tiện vận chuyển dựa theo thời gian thực. Bản đồ tích hợp các điểm đánh dấu thể hiện trạng thái, tuyến đường di chuyển, và các cảnh báo. Người dùng có thể tương tác trực tiếp với bản đồ để xem thêm thông tin chi tiết về từng phương tiện hoặc tuyến vận chuyển.

4



Hình 4.5.3: Các giao diện liên quan đến trang *Maps*

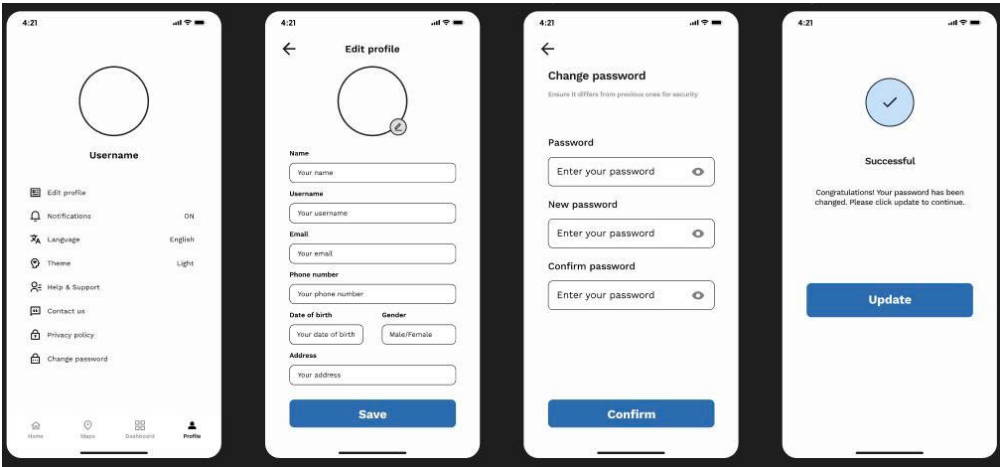
Trong Hình 4.5.4, giao diện Dashboard được thể hiện, cung cấp các biểu đồ thống kê và thông số quan trọng, hoặc các thông tin phân tích theo thời gian. Mục tiêu của dashboard là hỗ trợ người vận hành đưa ra quyết định nhanh chóng dựa trên dữ liệu trực quan.

4



Hình 4.5.4: Các giao diện liên quan đến trang *Dashboard*

Cuối cùng, Hình 4.5.5 mô tả trang Profile, nơi người dùng có thể xem và chỉnh sửa thông tin cá nhân, thay đổi cài đặt tài khoản, hoặc đăng xuất khỏi hệ thống. Giao diện được thiết kế đơn giản, tập trung vào sự rõ ràng và dễ thao tác.



Hình 4.5.5: Các giao diện liên quan đến trang *Profile*

CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

5

5.1 Cài đặt môi trường phát triển

5.1.1 Môi trường phát triển phần cứng

5.1.1.1 Môi trường phát triển firmware LoRa P2P

Do module Ra-08H sử dụng SoC ASR6601 dựa trên kiến trúc ARM Cortex-M4, quá trình phát triển firmware cho hệ thống LoRa P2P yêu cầu bộ công cụ chuyên biệt do nhà sản xuất cung cấp. Trong quá trình hiện thực, nhóm sử dụng môi trường phát triển Keil μ Vision 5 (MDK-ARM) để biên dịch và quản lý mã nguồn ngôn ngữ C cho vi điều khiển. Đây là công cụ phổ biến trong phát triển hệ thống nhúng, hỗ trợ tốt cho kiến trúc ARM và tích hợp đầy đủ các chức năng build, debug và quản lý bộ nhớ.

Bên cạnh đó, ASR6601 SDK được sử dụng làm nền tảng phát triển firmware, cung cấp các thư viện driver phần cứng, cấu hình radio LoRa và các ví dụ mẫu, trong đó có chương trình pingpong được nhóm lựa chọn làm cơ sở để phát triển giao thức truyền thông LoRa P2P. Việc sử dụng SDK chính hãng giúp đảm bảo tính tương thích, ổn định và rút ngắn thời gian triển khai hệ thống.

Để thuận tiện cho việc tạo và cấu hình dự án, nhóm đã sử dụng công cụ hỗ trợ mang tên KeilProjectGen, là một script hỗ trợ sinh tự động file dự án .uvprojx từ cấu trúc mã nguồn của SDK. Công cụ này giúp giảm thiểu các lỗi cấu hình thủ công và đảm bảo sự đồng nhất giữa các project firmware. Cuối cùng, quá trình nạp firmware vào bộ nhớ flash của chip ASR6601 được thực hiện thông qua phần mềm TremoProg phiên bản 1.0, cho phép nạp file binary

trực tiếp vào module Ra-08H một cách nhanh chóng và ổn định.

5.1.2 Môi trường phát triển phần mềm

Môi trường phát triển phần mềm của hệ thống được xây dựng nhằm đáp ứng đồng thời các yêu cầu về lập trình vi điều khiển, triển khai backend xử lý dữ liệu và phát triển ứng dụng di động.

- **Vi điều khiển (ESP32):** Nhóm sử dụng Visual Studio Code kết hợp với Extension PlatformIO trên nền tảng Framework Arduino. Cách tiếp cận này giúp đơn giản hóa quá trình quản lý thư viện, biên dịch và nạp chương trình, đồng thời tăng khả năng mở rộng và bảo trì mã nguồn.
- **Backend & Middleware:** Ngôn ngữ lập trình Python phiên bản 3.x được sử dụng để xây dựng script trung gian (Bridge) thực hiện chức năng kết nối giữa MQTT Broker và cơ sở dữ liệu Firebase. Việc lựa chọn Python cho phép phát triển nhanh, dễ dàng xử lý dữ liệu dạng JSON và tích hợp hiệu quả với các thư viện hỗ trợ như firebase-admin. Công tác quản lý và giám sát dữ liệu được thực hiện thông qua Google Firebase Console, đóng vai trò là nền tảng lưu trữ và đồng bộ dữ liệu thời gian thực cho hệ thống.
- **Mobile Application:** Nhóm sử dụng Flutter SDK phiên bản mới nhất để phát triển ứng dụng đa nền tảng, kết hợp với Android Studio nhằm hỗ trợ giả lập, kiểm thử và đóng gói ứng dụng dưới dạng file APK. Ngoài ra, hệ thống sử dụng phần mềm Mosquitto MQTT được cài đặt trên hệ điều hành Windows để làm MQTT Broker, chịu trách nhiệm quản lý kết nối, phân phối và truyền tải dữ liệu giữa các thành phần trong hệ thống.

5.2 Cài đặt firmware cho Node Lora P2P

5.2.1 Các bước build firmware LoRa P2P

Quy trình biên dịch và nạp firmware cho module Ra-08H được thực hiện theo một chuỗi các bước tuần tự nhằm đảm bảo firmware được cấu hình đúng cho SoC ASR6601 và hoạt động ổn định trong môi trường thực tế. Trước hết, nhóm sử dụng mã nguồn ví dụ pingpong được cung cấp trong ASR6601 SDK làm nền tảng phát triển, sau đó tiến hành tùy chỉnh và mở rộng chức năng để xây dựng phiên bản pingpong_uart. Phiên bản này được bổ sung cơ chế giao tiếp UART, cho phép module LoRa trao đổi dữ liệu trực tiếp với vi điều khiển trung tâm ESP32.

Sau khi hoàn tất việc chỉnh sửa mã nguồn, project firmware được tạo tự động thông qua công cụ KeilProjectGen. Script này có nhiệm vụ sinh file dự án

Keil với các thiết lập phù hợp cho kiến trúc ASR6601, giúp giảm thiểu các sai sót trong quá trình cấu hình thủ công. Project sau đó được mở bằng môi trường Keil μ Vision 5 để thực hiện quá trình biên dịch. Khi quá trình build hoàn tất thành công, hệ thống sẽ tạo ra file nhị phân pingpong.bin, là đầu vào cho bước nạp firmware.

Ở bước cuối cùng, firmware được nạp vào bộ nhớ flash của module Ra-08H thông qua phần mềm TremoProg. Để đưa module vào chế độ nạp, chân BOOT0 được kéo lên mức logic cao (3.3V) và module được reset. Sau đó, người vận hành lựa chọn đúng cổng COM, chỉ định file .bin tương ứng và thực hiện lệnh nạp. Khi quá trình này kết thúc thành công, module sẵn sàng hoạt động với firmware LoRa P2P đã được triển khai.

5.2.2 *Quá trình phát triển firmware LoRa P2P*

Nhóm đã phát triển hai phiên bản firmware riêng biệt cho Node gửi và Node nhận dựa trên giao thức LoRa P2P (Peer-to-Peer).

5

5.2.2.1 *Node TX (Truyền tin)*

Firmware của Node TX được xây dựng với mục tiêu đóng vai trò như một bộ chuyển đổi dữ liệu từ giao tiếp UART sang kênh truyền không dây LoRa (UART-to-LoRa). Trong giai đoạn khởi tạo, cổng UART1 của module Ra-08H được cấu hình với tốc độ truyền 115200 bps nhằm đảm bảo khả năng giao tiếp ổn định và phù hợp với vi điều khiển ESP32. Việc lựa chọn baudrate này giúp cân bằng giữa tốc độ truyền dữ liệu và độ ổn định của hệ thống trong điều kiện vận hành thực tế.

Để xử lý dữ liệu nhận được từ ESP32, firmware sử dụng cơ chế lưu trữ tạm thời thông qua bộ đệm. Kích thước bộ đệm BUFFER_SIZE được mở rộng lên 256 bytes nhằm đảm bảo có thể chứa đầy đủ chuỗi dữ liệu dạng JSON được gửi từ các cảm biến trong mỗi chu kỳ đo. Cách tiếp cận này giúp hạn chế hiện tượng tràn bộ đệm và đảm bảo tính toàn vẹn của dữ liệu trước khi tiến hành truyền không dây.

Về mặt logic hoạt động, vi xử lý liên tục giám sát dữ liệu trên cổng UART. Khi phát hiện ký tự kết thúc dòng ($\backslash n$), firmware xác định rằng một gói dữ liệu hoàn chỉnh đã được nhận. Toàn bộ nội dung trong bộ đệm sau đó được đóng gói thành payload LoRa và được truyền đi bằng cách gọi hàm Radio.Send(). Cơ chế này cho phép Node TX thực hiện truyền dữ liệu một cách tuần tự, đảm bảo mỗi gói tin LoRa chứa đầy đủ thông tin cần thiết trước khi phát sóng. Đoạn mã dưới đây trích xuất từ file pingpong_tx.c, minh họa logic chính trong vòng

lập vô hạn của Node TX. Vì xử lý liên tục giám sát dữ liệu trên cổng UART, đọc từng byte vào bộ đệm line. Khi phát hiện ký tự kết thúc dòng (\n) hoặc khi bộ đệm đầy, firmware sẽ gọi hàm `Radio.Send()` để phát sóng gói tin LoRa đi:

```

1  while (1) {
2      while (uart_available_wrapper() && idx < (int)sizeof(line)-1) {
3          int c = uart_readbyte_wrapper();
4          if (c < 0) break;
5
6  #if UART0_ECHO
7          uart_writebyte((uint8_t)c);
8  #endif
9          line[idx++] = (uint8_t)c;
10         last_rx_tick = TimerGetCurrentTime();
11         if (c == '\n') break;
12     }
13
14     if (idx > 0) {
15         int have_nl = (line[idx-1] == '\n');
16         int idle = (TimerGetElapsedTime(last_rx_tick) > 80);
17
18         if (have_nl || (idle && idx > 0)) {
19             Radio.Send(line, (uint8_t)idx);
20             printf("Sent %d bytes\r\n", idx);
21
22             /* Reset buffer */
23             idx = 0;
24             memset(line, 0, sizeof(line));
25         }
26     }
27     Radio.IrqProcess();
28 }

```

Listing 5.1: Logic đọc UART và gửi LoRa tại Node TX

5.2.2.2 Node RX (Gateway nhận tin)

Firmware của Node RX được thiết kế để hoạt động như một Gateway trung gian, thực hiện chức năng chuyển đổi dữ liệu từ kênh truyền không dây LoRa sang giao tiếp UART (LoRa-to-UART). Node RX đóng vai trò là điểm thu thập dữ liệu từ các Node TX trong hệ thống và chuyển tiếp thông tin này tới vi điều khiển trung tâm hoặc máy tính để xử lý ở các tầng cao hơn.

Khi module LoRa nhận được một gói tin hợp lệ, sự kiện nhận được kích hoạt thông qua hàm callback `OnRxDone` do thư viện Radio cung cấp. Tại thời điểm này, firmware tiến hành trích xuất payload dữ liệu gốc từ gói tin nhận được, đồng thời thu thập các thông số đặc trưng của kênh truyền bao gồm cường độ tín hiệu thu (RSSI) và tỷ lệ tín hiệu trên nhiễu (SNR). Các thông số này phản ánh chất lượng liên kết vô tuyến và đóng vai trò quan trọng trong việc đánh giá hiệu năng truyền thông của hệ thống.

Sau khi hoàn tất quá trình xử lý và làm giàu dữ liệu, firmware tiến hành đóng gói payload LoRa cùng với các giá trị RSSI và SNR thành một chuỗi dữ liệu dạng JSON. Chuỗi JSON này sau đó được gửi ra cổng UART để chuyển tiếp

tới ESP32 hoặc máy tính phục vụ cho các bước xử lý tiếp theo, lưu trữ và hiển thị dữ liệu trong hệ thống. Cách tiếp cận này cho phép tách biệt rõ ràng giữa tầng truyền thông không dây và tầng xử lý ứng dụng, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và phân tích hiệu năng hệ thống trong các giai đoạn tiếp theo. Đoạn mã sau thể hiện quá trình xử lý khi nhận được gói tin (State = RX). Dữ liệu payload được mã hóa sang Hex để đảm bảo an toàn, sau đó được đóng gói cùng với RSSI, SNR, tần số và timestamp thành một chuỗi JSON hoàn chỉnh trước khi gửi qua UART:

```

1 switch (State)
2 {
3     case RX: {
4         char hex[(BUFFER_SIZE*2)+1];
5         hex_encode(RxBuf, RxLen, hex);
6
7         char json[BUFFER_SIZE*3 + 128];
8         int n = snprintf(json, sizeof(json),
9             "{ \"ts\":%lu, \"freq\":%lu, \"rssi\":%d, \"snr\":%d, \"len\":%u, \"payload_hex\": \"%s\" } \n",
10             (unsigned long)TimerGetCurrentTime(),
11             (unsigned long)RF_FREQUENCY,
12             (int)RxRssi, (int)RxSnr,
13             (unsigned)RxLen, hex);
14         if (n > 0) uart0_write((const uint8_t*)json, n);
15
16         Radio.Rx(RX_TIMEOUT_VALUE);
17         State = LOWPOWER;
18         break;
19     }
20
21     case RX_TIMEOUT:
22     case RX_ERROR:
23         Radio.Rx(RX_TIMEOUT_VALUE);
24         State = LOWPOWER;
25         break;
26
27     // ...
28 }

```

Listing 5.2: Logic đóng gói JSON và gửi UART tại Node RX

Cách tiếp cận này cho phép tách biệt rõ ràng giữa tầng truyền thông không dây và tầng xử lý ứng dụng, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và phân tích hiệu năng hệ thống trong các giai đoạn tiếp theo.

5.3 Cài đặt backend và xử lý dữ liệu

5.3.1 Thiết lập hạ tầng MQTT Broker

Hệ thống sử dụng Mosquitto làm MQTT Broker đóng vai trò trung tâm trong việc định tuyến và phân phối dữ liệu. Để đảm bảo khả năng truy cập từ các thiết bị ngoại vi trong mạng nội bộ, cấu hình của Broker đã được tùy chỉnh để lắng nghe trên tất cả các giao diện mạng (0.0.0.0) tại cổng tiêu chuẩn 1883, thay vì chỉ giới hạn ở localhost như mặc định. Đồng thời, các quy tắc tường lửa (Inbound Rules) trên máy chủ Windows cũng được thiết lập để cho phép các

kết nối TCP đến cổng này, đảm bảo luồng dữ liệu từ Gateway đến Broker được thông suốt.

5.3.2 Triển khai dịch vụ cầu nối (Bridge Service)

Để kết nối giữa luồng dữ liệu thời gian thực từ MQTT và cơ sở dữ liệu lưu trữ lâu dài Firebase, nhóm đã phát triển một dịch vụ trung gian (Bridge Service) bằng ngôn ngữ Python. Dịch vụ này không chỉ đơn thuần là chuyển tiếp dữ liệu mà còn thực hiện chức năng chuẩn hóa và làm giàu dữ liệu (Data Enrichment).

Cụ thể, script Python thực hiện đăng ký (subscribe) các chủ đề (topic) dữ liệu từ Gateway thông qua wildcard `ammo_transport/+/telemetry`. Khi nhận được gói tin, script tiến hành phân tích cú pháp JSON, trích xuất các trường thông tin quan trọng và thực hiện chuyển đổi định dạng. Dữ liệu thô từ cảm biến được kết hợp với các thông số kỹ thuật của mạng LoRa (RSSI, SNR) để tạo thành bản ghi hoàn chỉnh.

Dữ liệu sau xử lý được ghi vào Cloud Firestore theo mô hình hai lớp: lớp *Latest State* phục vụ cho việc hiển thị trạng thái tức thời trên Dashboard, và lớp *History* lưu trữ chuỗi thời gian (time-series) phục vụ cho nhu cầu truy xuất lịch sử và phân tích xu hướng. Việc sử dụng thư viện `firebase-admin` với cơ chế xác thực qua Service Account đảm bảo tính bảo mật và quyền kiểm soát truy cập dữ liệu nghiêm ngặt.

5.4 Cài đặt và phát triển ứng dụng di động

5.4.1 Kiến trúc ứng dụng Flutter

Ứng dụng di động được xây dựng dựa trên Flutter SDK, áp dụng kiến trúc phân tầng rõ ràng để tách biệt giữa giao diện người dùng (UI), logic nghiệp vụ (Business Logic) và các dịch vụ nền tảng (Services). Mã nguồn được tổ chức thành các module chức năng riêng biệt:

- **Module Giao diện (Screens & Components):** Bao gồm các màn hình tương tác chính và các widget tái sử dụng. Các thành phần này chỉ chịu trách nhiệm hiển thị dữ liệu và tiếp nhận thao tác người dùng, không chứa logic xử lý phức tạp.
- **Module Dịch vụ (Services):** Tập trung các lớp xử lý kết nối mạng, quản lý trạng thái và tương tác với API bên ngoài. Điển hình là `MqttService`, chịu trách nhiệm duy trì kết nối bền vững với Broker và xử lý các sự kiện ngắt kết nối hoặc lỗi mạng.
- **Module Mô hình dữ liệu (Models):** Định nghĩa cấu trúc dữ liệu ánh xạ

từ các bản tin JSON, giúp đảm bảo tính nhất quán và an toàn kiểu dữ liệu (type safety) trong toàn bộ ứng dụng.

5.4.2 Cơ chế đồng bộ dữ liệu thời gian thực

Thay vì sử dụng phương pháp hỏi vòng (polling) truyền thống gây tốn băng thông và năng lượng, ứng dụng áp dụng cơ chế lắng nghe sự kiện (event-driven) thông qua giao thức MQTT. Dịch vụ MqttService đăng ký theo dõi các chủ đề dữ liệu liên quan, cho phép ứng dụng nhận được cập nhật ngay lập tức khi có gói tin mới từ thiết bị giám sát. Dữ liệu nhận về được phân tích và cập nhật trực tiếp vào trạng thái ứng dụng (State Management), kích hoạt việc vẽ lại giao diện (re-build UI) để hiển thị thông tin mới nhất cho người dùng với độ trễ thấp nhất.

5.5 Kết nối và tích hợp thiết bị

Hệ thống bao gồm hai khối phần cứng chính: Node TX (Sensor Node) gắn trên phương tiện vận chuyển và Node RX (Gateway) đặt tại trạm giám sát. Cả hai đều sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm và module Ra-08H để truyền thông LoRa P2P.

5.5.1 Sơ đồ kết nối Node TX (Sensor Node)

Node TX chịu trách nhiệm thu thập dữ liệu từ đa cảm biến và gửi đi. Dựa trên mã nguồn firmware, sơ đồ đấu nối giữa ESP32 và các module ngoại vi được mô tả chi tiết trong Bảng 5.5.1.

Bảng 5.5.1: Sơ đồ chân kết nối phần cứng Node TX

Module	Giao tiếp	Chân ESP32	Chân Module
LoRa Ra-08H	UART1	GPIO 26 (TX)	RX
		GPIO 25 (RX)	TX
GPS NEO-6M	UART2	GPIO 17 (TX)	RX
		GPIO 16 (RX)	TX
DHT11	GPIO	GPIO 14	DATA
ADXL345	I2C	GPIO 21 (SDA)	SDA
		GPIO 22 (SCL)	SCL
Nguồn	Power	3.3V / GND	VCC / GND

Lưu ý cấu hình trong Firmware:

- **LoRa UART:** Sử dụng `HardwareSerial(1)` với tốc độ Baudrate là **115200** để đảm bảo đồng bộ với firmware của Ra-08H.
- **GPS UART:** Sử dụng tốc độ **9600** baud tiêu chuẩn của module NEO-6M.
- **Cảm biến gia tốc ADXL345:** Sử dụng giao tiếp I2C mặc định của ESP32 (Wire library).

5.5.2 Sơ đồ kết nối Node RX (Gateway Node)

Node RX đóng vai trò nhận dữ liệu LoRa và chuyển tiếp. Cấu hình phần cứng tối giản hơn, chỉ bao gồm ESP32 kết nối với module LoRa Ra-08H để thực hiện chức năng Gateway.

Bảng 5.5.2: Sơ đồ chân kết nối phần cứng Node RX

Module	Giao tiếp	Chân ESP32	Chân Module
LoRa Ra-08H	UART1	GPIO 26 (TX) GPIO 25 (RX)	RX TX
Máy tính (Server)	UART0 (USB)	USB Port	USB Port

5.5.3 Tích hợp dữ liệu (Data Integration)

Để đảm bảo tính toàn vẹn dữ liệu khi truyền giữa ESP32 và Ra-08H, hệ thống sử dụng định dạng chuỗi **JSON**. Trong tác vụ `TaskLoraSend` của ESP32, các dữ liệu cảm biến rời rạc được đóng gói thành một bản tin duy nhất trước khi gửi qua UART tới module LoRa.

Cấu trúc bản tin JSON được định nghĩa như sau:

```

1 snprintf(payload, sizeof(payload),
2  "{\"device_id\":\"Ra-08H-Node1\", \"
3  \"timestamp\":%lu, \"
4  \"temp\":%.2f, \"
5  \"battery\":%.2f, \"
6  \"gps\":{\"lat\":%.4f,\"lng\":%.4f}, \"
7  \"rssi_lora\":%d}\n",
8  timestamp, temp, battery, lat, lng, rssi);

```

Listing 5.3: Đoạn code đóng gói JSON trên Node TX

Việc sử dụng JSON giúp Node RX dễ dàng giải mã (parse) và Gateway có thể đẩy trực tiếp dữ liệu này lên MQTT Broker mà không cần xử lý định dạng phức tạp.

5.6 Quy trình vận hành và kiểm thử hệ thống

Hệ thống được vận hành theo quy trình khởi động tuần tự từ trung tâm ra ngoại vi (Center-to-Edge) để đảm bảo tính sẵn sàng của hạ tầng trước khi thiết bị đầu cuối hoạt động.

Bước 1: Khởi động hạ tầng máy chủ. Quá trình bắt đầu bằng việc kích hoạt dịch vụ Mosquitto MQTT Broker và chạy script Python Bridge. Tại bước này, các kết nối đến cơ sở dữ liệu Firebase được thiết lập và xác thực, đồng thời các cổng lắng nghe mạng được mở để sẵn sàng tiếp nhận kết nối.

Bước 2: Kích hoạt Gateway (Node RX). Gateway sau khi được cấp nguồn sẽ thực hiện quy trình tự kiểm tra (POST), kết nối vào mạng WiFi cục bộ và thiết lập phiên làm việc với MQTT Broker. Sau khi kết nối thành công, Gateway chuyển sang chế độ lắng nghe liên tục trên kênh tần số LoRa đã định.

Bước 3: Vận hành thiết bị giám sát (Node TX). Thiết bị giám sát gắn trên phương tiện vận tải được khởi động cuối cùng. Vi điều khiển ESP32 bắt đầu chu kỳ thu thập dữ liệu từ các cảm biến GPS, nhiệt độ và gia tốc kế. Dữ liệu được xử lý, đóng gói và truyền đi định kỳ qua module LoRa.

Bước 4: Giám sát và Điều hành. Người dùng cuối truy cập vào hệ thống thông qua ứng dụng di động. Tại đây, luồng dữ liệu từ hiện trường được trực quan hóa thành các thông số giám sát và vị trí trên bản đồ số. Hệ thống cho phép người điều hành theo dõi trạng thái vận chuyển theo thời gian thực và nhận các cảnh báo tức thời khi có bất thường xảy ra.

Bảng 5.6.1: Tổng hợp chức năng và vai trò của các thành phần trong hệ thống

Thành phần	Vai trò kỹ thuật và Chức năng
Node TX	Thu thập biên (Edge Acquisition): Đo đạc thông số môi trường và vận hành, xử lý tín hiệu sơ bộ và truyền tải không dây tầm xa.
Node RX	Cổng kết nối (Gateway): Chuyển đổi môi trường truyền tin từ sóng vô tuyến (RF) sang mạng IP, đóng vai trò điểm tập trung dữ liệu.
MQTT Broker	Trục truyền tin (Message Bus): Đảm bảo phân phối dữ liệu tin cậy, thời gian thực giữa các thành phần phân tán.
Python Bridge	Xử lý trung gian (Middleware): Chuẩn hóa dữ liệu, tích hợp logic nghiệp vụ và đồng bộ hóa với hệ thống lưu trữ đám mây.
Firestore	Lưu trữ đám mây (Cloud Storage): Quản lý dữ liệu bền vững, hỗ trợ truy vấn lịch sử và đồng bộ trạng thái đa thiết bị.
Mobile App	Giao diện người dùng (HMI): Trực quan hóa dữ liệu giám sát, cung cấp công cụ tương tác và ra quyết định cho người quản lý.

6

KẾT QUẢ VÀ ĐÁNH GIÁ

6.1 Các kịch bản thử nghiệm

Để đánh giá toàn diện hiệu năng của hệ thống, nhóm thực hiện chuỗi các kịch bản kiểm thử từ môi trường phòng thí nghiệm đến hiện trường thực tế. Các kịch bản được thiết kế để kiểm chứng khả năng giám sát vị trí, độ ổn định truyền thông và độ tin cậy của hệ thống cảnh báo.

6.1.1 Kịch bản kiểm thử khoảng cách truyền thông (Range Test)

Kịch bản này được thiết kế để xác định giới hạn truyền dẫn của module Ra-08H trong môi trường đô thị phức tạp (Khu vực KTX Khu A - ĐHQG TP.HCM), đặc trưng bởi mật độ bê tông cốt thép dày đặc gây suy hao tín hiệu lớn (Non-Line-of-Sight - NLOS).

Thử nghiệm được tiến hành tại ba mốc khoảng cách: 100m (tầm nhìn thẳng - LOS), 500m (NLOS) và 1000m (NLOS). Tại mỗi mốc, nhóm thực hiện đo đạc song song với hai cấu hình Spreading Factor: $SF = 7$ (ưu tiên tốc độ truyền) và $SF = 11$ (ưu tiên độ nhạy thu và khả năng chống nhiễu).

6.1.1.1 Kết quả thử nghiệm

Kết quả thực nghiệm về tỷ lệ mất gói tin (Packet Loss Rate - PLR), cường độ tín hiệu (RSSI) và tỷ lệ tín hiệu trên nhiễu (SNR) được tổng hợp trong Bảng 6.1.1.

Khoảng cách	Môi trường	Cấu hình SF = 7		Cấu hình SF = 11	
		PLR	RSSI/SNR	PLR	RSSI/SNR
100m	LOS	0%	-73 dBm / 5 dB	0%	-80 dBm / 8.5 dB
500m	NLOS	>95% (Fail)	N/A	5%	-105 dBm / 3.1 dB
1000m	NLOS	100% (Fail)	N/A	5%	-73dBm / 3.1dB

Bảng 6.1.1: Kết quả đo đạc Range Test tại KTX Khu A

6.1.1.2 Phân tích và Đánh giá

Dựa trên các số liệu thực nghiệm thu được, có thể rút ra một số nhận xét quan trọng về độ ổn định truyền thông của hệ thống LoRa sử dụng module Ra-08H trong các điều kiện khoảng cách và môi trường khác nhau.

Ở khoảng cách gần 100 mét trong điều kiện tầm nhìn thẳng (LOS), cả hai cấu hình Spreading Factor $SF = 7$ và $SF = 11$ đều cho kết quả hoạt động ổn định với tỷ lệ mất gói tin bằng 0%. Cường độ tín hiệu thu được ở mức rất tốt, với giá trị RSSI dao động từ khoảng -73 dBm đến -80 dBm. Trong phạm vi này, cấu hình $SF = 7$ được đánh giá là phù hợp hơn do có thời gian truyền (air-time) ngắn, giúp giảm độ trễ và tiết kiệm năng lượng cho thiết bị đầu cuối.

Tại khoảng cách trung bình 500 mét trong điều kiện không có tầm nhìn thẳng (NLOS), sự khác biệt rõ rệt giữa hai cấu hình Spreading Factor bắt đầu thể hiện. Với cấu hình $SF = 7$, hệ thống gần như không duy trì được kết nối do tín hiệu bị suy hao mạnh khi truyền qua các khối nhà bê tông. Ngược lại, cấu hình $SF = 11$ vẫn đảm bảo khả năng truyền thông tương đối ổn định với tỷ lệ mất gói tin chỉ khoảng 5%. Giá trị RSSI trung bình đo được vào khoảng -105 dBm, cho thấy khả năng xuyên vật cản và độ nhạy thu vượt trội của cấu hình SF cao. Kết quả này chứng minh rằng $SF = 11$ là lựa chọn phù hợp hơn đối với các ứng dụng giám sát vận chuyển trong môi trường đô thị phức tạp.

Khi khoảng cách truyền tăng lên 1000 mét trong điều kiện NLOS, hệ thống xuất hiện hiện tượng timeout liên tục ở cả hai cấu hình $SF = 7$ và $SF = 11$, dẫn đến việc mất kết nối hoàn toàn. Nguyên nhân chính được xác định là do mật độ bê tông cốt thép dày đặc tại khu vực thử nghiệm, khiến suy hao tín hiệu vượt quá ngưỡng độ nhạy thu của module Ra-08H (khoảng -140 dBm). Để cải thiện khả năng truyền thông trong trường hợp này, các giải pháp như nâng cao vị trí đặt anten, sử dụng anten có độ lợi (gain) cao hơn so với anten lò xo mặc định, hoặc triển khai thêm các gateway trung gian cần được xem xét trong các giai đoạn phát triển tiếp theo.

6.1.2 Kịch bản kiểm thử chức năng cảnh báo

Kịch bản kiểm thử chức năng cảnh báo được xây dựng nhằm mô phỏng các tình huống nguy hiểm có thể xảy ra trong quá trình vận chuyển đạn dược, từ đó đánh giá độ nhạy và độ trễ của hệ thống cảnh báo. Mục tiêu của kịch bản này là kiểm chứng khả năng phát hiện kịp thời các sự cố bất thường cũng như tính chính xác của thông báo cảnh báo được gửi đến ứng dụng di động và Dashboard giám sát.

Thiết lập ngưỡng cảnh báo

Dựa trên các thông số an toàn cho việc vận chuyển đạn dược, các ngưỡng cảnh báo được thiết lập cứng trong firmware như sau:

- **Cảnh báo nhiệt độ:** Kích hoạt khi $T > 30.0^{\circ}\text{C}$ và chỉ giải tỏa cảnh báo khi $T < 29.0^{\circ}\text{C}$.
- **Cảnh báo độ ẩm:** Kích hoạt khi $H > 80.0\%$ và giải tỏa khi $H < 75.0\%$.
- **Cảnh báo va chạm (Shock):** Kích hoạt khi độ lớn gia tốc tổng hợp $|a| > 2g$.

Trong quá trình thử nghiệm, nhóm tiến hành xây dựng ba tình huống giả định đại diện cho các rủi ro thường gặp trong thực tế. Thứ nhất, đối với kịch bản cảnh báo nhiệt độ cao, máy sấy nhiệt được sử dụng để tác động trực tiếp lên cảm biến DHT11, làm nhiệt độ môi trường xung quanh thiết bị tăng dần và vượt qua ngưỡng cài đặt trên hệ thống. Thứ hai, kịch bản cảnh báo va đập và rung lắc được thực hiện bằng cách tạo ra các rung động mạnh đột ngột lên thiết bị, nhằm kích hoạt cảm biến gia tốc ADXL345 khi gia tốc đo được vượt quá ngưỡng $2G$. Cuối cùng, đối với kịch bản cảnh báo pin yếu, nhóm sử dụng nguồn cấp giả lập với điện áp giảm dần cho đến khi thấp hơn mức 3.3V , từ đó kiểm tra khả năng phát hiện và phát cảnh báo của hệ thống khi nguồn năng lượng không còn đảm bảo.

Thông qua các kịch bản kiểm thử này, hệ thống được đánh giá toàn diện về khả năng nhận diện sự cố, thời gian phản hồi cũng như độ tin cậy của cơ chế cảnh báo trong điều kiện vận hành thực tế.

6.1.2.1 Kết quả thử nghiệm

6.1.2.2 Phân tích và đánh giá

6.2 Kiểm thử tích hợp và Phân tích độ trễ

6.2.1 Phân rã độ trễ toàn trình (Latency Breakdown)

Đối với các hệ thống cảnh báo nguy hiểm, độ trễ truyền tin là yếu tố sống còn. Nhóm đã thực hiện đo đạc và phân rã độ trễ toàn trình (End-to-End Latency) để xác định các điểm nghẽn trong hệ thống. Độ trễ tổng (T_{total}) được tính bằng tổng thời gian xử lý tại các chặng:

$$T_{total} = T_{sensor} + T_{LoRa} + T_{Gateway} + T_{Cloud} + T_{App}$$

Kết quả đo đạc trung bình trên 100 mẫu thử với cấu hình $SF = 11$ được trình bày trong Bảng 6.2.1.

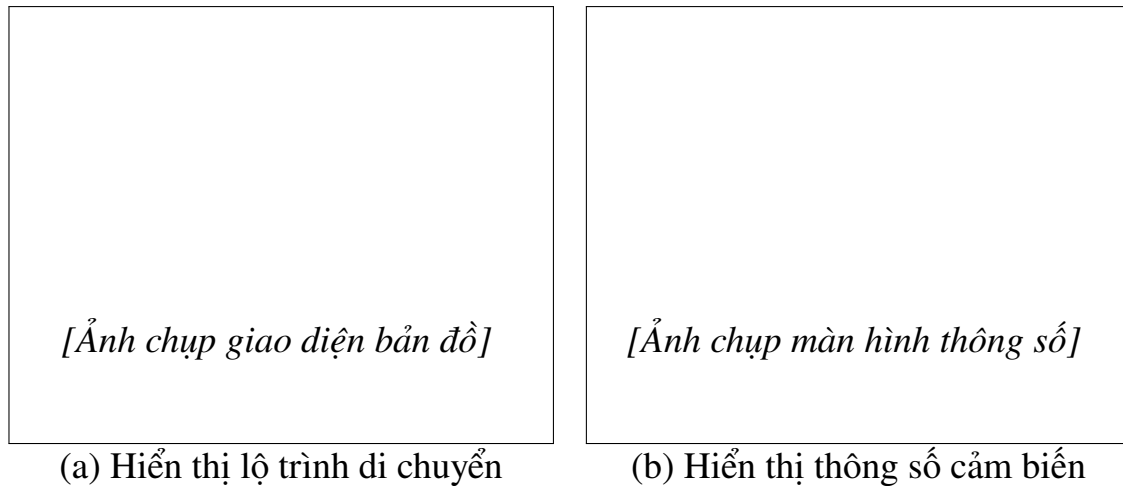
Bảng 6.2.1: Phân rã độ trễ xử lý dữ liệu của hệ thống

Thành phần	Mô tả quá trình	Thời gian (ms)
Sensor Node	Đọc cảm biến, lọc nhiễu, đóng gói JSON	50
LoRa Link	Thời gian truyền sóng (Air-time, SF11)	$\approx 800 - 1000$
Gateway	Nhận gói tin, giải mã, đẩy lên MQTT Broker	100
Cloud Bridge	Python script xử lý, ghi vào Firestore	200 - 500
Mobile App	Listener nhận event và cập nhật UI	50 - 100
Tổng cộng	Độ trễ toàn trình trung bình	$\approx 1.2s - 2.5s$

Kết luận: Độ trễ trung bình khoảng 2 giây là chấp nhận được đối với bài toán giám sát vận tải (Soft Real-time). Thành phần chiếm tỷ trọng lớn nhất là T_{LoRa} (do sử dụng SF cao để tăng tầm phủ sóng) và T_{Cloud} (phụ thuộc vào tốc độ mạng Internet).

6.2.2 Thử nghiệm mô phỏng vận chuyển thực tế

Hệ thống đã được triển khai thử nghiệm trên xe máy di chuyển trong khuôn viên Đại học Quốc gia TP.HCM. Gateway được đặt cố định tại tòa nhà H6. Ứng dụng di động đã hiển thị chính xác lộ trình di chuyển (tracking path) trùng khớp với các tuyến đường trên bản đồ số Google Maps, chứng tỏ thuật toán chuyển đổi tọa độ GPS sang GeoPoint của Firestore hoạt động chính xác.



Hình 6.2.1: *Giao diện ứng dụng di động trong quá trình thử nghiệm thực tế*

6.3 Đánh giá kết quả đạt được

Trải qua quá trình nghiên cứu, thiết kế và thực nghiệm, đề tài đã hoàn thành các mục tiêu ban đầu đề ra, xây dựng được hệ thống giám sát vận chuyển đạn được dựa trên công nghệ IoT. Nhìn chung, các kết quả đạt được có thể tóm tắt như sau:

6

- **Về phần cứng:** Đã thiết kế và tích hợp thành công các node cảm biến thu thập đa thông số (vị trí, nhiệt độ, độ ẩm, va chạm) sử dụng vi điều khiển trung tâm ESP32 và module truyền thông LoRa Ra-08H. Hệ thống hoạt động ổn định trong các kịch bản thực tế tại khu vực KTX Khu A ĐHQG-HCM.
- **Về truyền thông:** Công nghệ LoRa P2P với cấu hình $SF = 11$ đã chứng minh được tính hiệu quả trong môi trường đô thị nhiều vật cản. Thử nghiệm thực tế cho thấy hệ thống duy trì kết nối tốt ở khoảng cách 500m NLOS với tỷ lệ mất gói thấp (khoảng 5%), vượt trội hơn hẳn so với các giải pháp WiFi/Zigbee truyền thống trong cùng điều kiện.
- **Về phần mềm và hệ thống:** Xây dựng thành công lớp trung gian (Middleware) bằng Python Bridge kết hợp MQTT Broker, giúp chuẩn hóa và làm giàu dữ liệu (RSSI/SNR) trước khi lưu trữ vào Cloud Firestore. Ứng dụng di động phản hồi dữ liệu thời gian thực với độ trễ từ 1.2 giây đến 2.5 giây, đáp ứng yêu cầu giám sát hành trình.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Đánh giá chung

Trong phạm vi nghiên cứu, nhóm đã thiết kế và triển khai thành công hệ thống IoT giám sát và điều phối vận chuyển đạn dược ở mức cơ bản dựa trên nền tảng vi điều khiển ESP32 kết hợp công nghệ truyền thông LoRa P2P và hạ tầng xử lý dữ liệu trung tâm thông qua MQTT và Firebase. Hệ thống hướng đến mục tiêu nâng cao mức độ an toàn, khả năng giám sát theo thời gian thực và hỗ trợ điều phối hiệu quả trong quá trình vận chuyển các vật tư quân sự có độ nhạy cảm cao.

Về mặt kỹ thuật, hệ thống đã tích hợp đầy đủ các thành phần phần cứng cần thiết bao gồm cảm biến nhiệt độ – độ ẩm DHT11, cảm biến gia tốc ADXL345 và module GPS NEO-6MV2, cho phép thu thập liên tục các thông số môi trường và trạng thái vận chuyển. Dữ liệu sau khi được mã hóa được truyền qua LoRa đến Gateway, sau đó được chuyển tiếp lên hệ thống trung tâm thông qua MQTT Broker và hiển thị trực quan trên ứng dụng giám sát.

Các kết quả thử nghiệm trong Chương 6 cho thấy hệ thống hoạt động ổn định trong điều kiện mô phỏng, khả năng truyền thông LoRa đạt phạm vi phù hợp với yêu cầu giám sát tầm xa, dữ liệu GPS có độ chính xác chấp nhận được và các chức năng cảnh báo được kích hoạt đúng theo ngưỡng thiết lập. Giao diện người dùng trên ứng dụng di động cho phép theo dõi vị trí, trạng thái cảm biến và lịch sử vận chuyển một cách trực quan, đáp ứng tốt các yêu cầu chức năng đã đề ra.

Nhìn chung, hệ thống đã đạt được các mục tiêu nghiên cứu ban đầu, chứng

minh tính khả thi của việc ứng dụng IoT trong bài toán giám sát và điều phối vận chuyển đạn dược, đồng thời tạo tiền đề cho việc mở rộng và hoàn thiện trong các nghiên cứu và triển khai thực tế sau này.

7.2 Hạn chế

Mặc dù hệ thống IoT giám sát và điều phối vận chuyển đạn dược đã đạt được các mục tiêu đề ra ban đầu, đề tài vẫn tồn tại một số hạn chế nhất định do điều kiện triển khai thực tế và giới hạn về nguồn lực.

Thứ nhất, việc kiểm thử khoảng cách truyền thông của công nghệ LoRa còn bị giới hạn. Các thí nghiệm truyền tín hiệu chủ yếu được thực hiện trong phạm vi hẹp (khu vực phòng thí nghiệm và khuôn viên ký túc xá), chưa thể đánh giá đầy đủ hiệu năng của LoRa trong các điều kiện thực tế như địa hình phức tạp, khoảng cách xa hàng chục kilomet hoặc môi trường có nhiều vật cản. Điều này khiến kết quả đo chỉ mang tính tham khảo và chưa phản ánh toàn diện khả năng triển khai trong thực tế quy mô lớn.

Thứ hai, nhóm gặp khó khăn trong việc mô phỏng đầy đủ các kịch bản truyền thông LoRa. Do thiếu các công cụ mô phỏng chuyên dụng cho LoRa ở tầng vật lý và tầng mạng, việc đánh giá các tình huống như nhiễu sóng, mất gói, va chạm kênh hay hoạt động đồng thời của nhiều node vẫn còn hạn chế. Phần lớn các thử nghiệm được thực hiện theo phương pháp thủ công, chưa thể tự động hóa hoặc mở rộng quy mô mô phỏng.

Thứ ba, do giới hạn về chi phí và thời gian, hệ thống mới dừng lại ở mức độ nguyên mẫu (prototype) và chưa được hoàn thiện thành một hệ thống triển khai thực tế. Các thành phần phần cứng chưa được tích hợp vào vỏ bảo vệ chuyên dụng, chưa đáp ứng các tiêu chuẩn công nghiệp hoặc quân sự về độ bền, chống rung và chống thời tiết. Ngoài ra, hạ tầng mạng, máy chủ và các cơ chế dự phòng cũng chưa được triển khai đầy đủ để đảm bảo khả năng vận hành liên tục trong thời gian dài.

Những hạn chế nêu trên là các yếu tố khách quan trong khuôn khổ của một đồ án tốt nghiệp, đồng thời cũng là cơ sở quan trọng để định hướng cho các nghiên cứu và phát triển tiếp theo trong tương lai.

7.3 Hướng phát triển

Dựa trên các kết quả đạt được và những hạn chế còn tồn tại, đề tài có thể được tiếp tục phát triển theo nhiều hướng nhằm nâng cao tính hoàn thiện và khả năng ứng dụng thực tế của hệ thống.

Trong giai đoạn tiếp theo, hệ thống có thể được nâng cấp từ mô hình LoRa

P2P sang LoRaWAN. Việc sử dụng LoRaWAN giúp quản lý tập trung các node, hỗ trợ xác thực thiết bị, mã hóa đầu cuối và khả năng mở rộng quy mô lớn với nhiều gateway. Điều này đặc biệt phù hợp khi triển khai hệ thống trong môi trường thực tế với nhiều phương tiện vận chuyển hoạt động đồng thời trên diện rộng.

Bên cạnh đó, ứng dụng giám sát cần được tối ưu cả về hiệu năng lẫn trải nghiệm người dùng. Các hướng cải tiến bao gồm tối ưu cơ chế đồng bộ dữ liệu thời gian thực, giảm độ trễ hiển thị, cải thiện giao diện trực quan trên bản đồ và dashboard, cũng như bổ sung các chức năng phân quyền và nhật ký hoạt động chi tiết cho từng vai trò người dùng.

Một hướng phát triển quan trọng khác là tích hợp các cơ chế bảo mật nâng cao và học máy. Hệ thống có thể được mở rộng với các thuật toán học máy nhằm phân tích dữ liệu lịch sử, phát hiện bất thường một cách thông minh hơn thay vì chỉ dựa trên ngưỡng cố định. Đồng thời, các kỹ thuật bảo mật nâng cao như quản lý khóa động, xác thực đa lớp và phát hiện tấn công bất thường cũng có thể được áp dụng để tăng mức độ an toàn cho hệ thống.

Cuối cùng, hệ thống có thể được kết hợp với các thuật toán tối ưu lộ trình vận chuyển. Dựa trên dữ liệu GPS, trạng thái giao thông và các ràng buộc an toàn, hệ thống có thể đề xuất hoặc tự động điều chỉnh lộ trình tối ưu nhằm giảm thời gian vận chuyển, hạn chế rủi ro và nâng cao hiệu quả điều phối. Hướng phát triển này giúp chuyển hệ thống từ vai trò giám sát thuần túy sang hỗ trợ ra quyết định thông minh trong công tác vận chuyển.

Các hướng phát triển nêu trên không chỉ giúp hoàn thiện hệ thống về mặt kỹ thuật mà còn mở ra khả năng ứng dụng thực tế rộng rãi hơn trong các bài toán giám sát và điều phối vận chuyển trong tương lai.

TÀI LIỆU THAM KHẢO